

LLM AI 개발 및 활용

리버스 엔지니어링과 SDD

AI 활용 로드맵 · 제품에서 명세 추출 및 재생성 · 화면 명세

접속 · 매주 같은 순서

실습 환경 접속과 점검

1주차에 갖춘 환경을 그대로 다시 엽니다. 처음이거나 절차를 잊었다면 다음 장부터.

1. 서버 접속

PowerShell에서 ssh 아이디@aisw02.tedso.kr (비밀번호는 쳐도 안 보임), 또는 VS Code Remote-SSH.
프롬프트가 [아이디@aisw02 ~]\$ 면 성공.

2. 터미널 2개 배치

서버 터미널과 Claude Code 터미널을 나란히 엽니다.

Claude Code 터미널은 지시하는 창,
서버 터미널은 결과를 직접 실행·검증하는 창입니다.

3. 환경을 점검합니다

claude 버전 → API 키 → 모델 순서로, 오른쪽 명령을 그대로 칩니다.

주의 지금 이 창이 내 PC인지 서버인지는 프롬프트가 알려줍니다.

PS C:\Users\이름> = 내 PC, [아이디@aisw02 ~]\$ = 서버

[서버 터미널]

```
$ claude --version
2.x.x (Claude Code)
$ echo ${ANTHROPIC_API_KEY:0:8}
sk-ant-a
```

[Claude Code 안]

```
> /status
Model: haiku
> /exit
```

비밀번호를 잊었으면 손을 들어 알립니다.

Model이 haiku가 아니면 /model에서 다시 선택합니다.

ssh로 서버 첫 접속: 내 PC에서 서버 안으로

1. PowerShell을 엽니다

시작 검색 "Windows PowerShell" 또는 Win+X → 터미널.

2. ssh 아이디@aisw02.tedso.kr

아이디 = 이름 영문 약자+성 (예: 권동해 → dhkwon).

3. 처음이면 yes 입력

continue connecting? 질문은 첫 접속에만 나옵니다.

4. 비밀번호 입력

쳐도 화면에 안 보이는 게 정상. 끊기면 다시 ssh부터.

5. 초기 비밀번호라면 passwd 로 변경

초기 비밀번호는 전원 공통이니 반드시 바꿉니다.

PowerShell → 서버 접속

```
PS C:\Users\이름> ssh 아이디@aisw02.tedso.kr
... continue connecting? (yes/no)? yes # 첫 접속에만
아이디@aisw02.tedso.kr's password: # 입력해도 안 보임
[아이디@aisw02 ~]$ # 접속 성공!
[아이디@aisw02 ~]$ passwd
# 현재 비밀번호 → 새 비밀번호 2번 → updated successfully
```

확인 포인트 프롬프트가 내 위치를 알려줍니다

PS C:\Users\이름> = 내 PC, [아이디@aisw02 ~]\$ = 서버.

곽정민 jmkwak · 권동해 dhkwon · 김유한 yhkim · 김하민 hmkim · 김현구 hgkim · 유인수 isyu

박승순 sspark · 손시울 syson · 엄지민 jmeom · 조원경 wgcho · 구민우 mugu · 허동진 djheo

설치와 모델 설정: 아직 안 했거나 잊었다면

이미 한 사람은 건너뛰니다. 자세한 절차는 1주차 강의안 4~6장.

1. 설치 (최초 1회)

서버 터미널에서 오른쪽 설치 명령 실행

→ 터미널을 닫았다 새로 열기

→ `claude --version` 확인.

2. API 키 확인·등록 (최초 1회)

키는 내 홈에 `.txt` 파일로 배포되어 있습니다.

`cat ~/.txt` 로 열어 복사하고, 등록 명령의 '발급키' 자리에 붙여 넣습니다.

3. 첫 실행 승인

claude 입력 → Yes → Security notes에서 Enter → 폴더 신뢰 Yes.

4. 모델을 Haiku로

기본값은 Opus 또는 Sonnet(버전에 따라 다름)이라 비쌉니다. 그대로 쓰지 말고 /model 에서 Haiku 선택 → /status 확인. 한 번 설정하면 유지됩니다.

[서버 터미널] · 설치와 키 등록

```
$ curl -fsSL https://claude.ai/install.sh | bash
$ claude --version
$ cat ~/.txt
  sk-ant-***** ← 복사
$ echo 'export ANTHROPIC_API_KEY=발급키' >> ~/.bashrc
$ source ~/.bashrc ← 등록한 키 바로 적용
```

[Claude Code 안] · 모델 설정

```
> /model → 목록에서 Haiku 선택 (Opus 금지)
> /status
  Model: haiku
> /exit
```

로그인 방법 선택 화면이 나오면 키 등록이 빠진 것입니다

Go back으로 나가 2번부터 다시.

설치·키 등록·모델 설정은 모두 최초 1회입니다. 재접속해도 유지됩니다.

작업 폴더 · 배포 폴더: 이번 주 폴더를 내 폴더로 복사

내 폴더 /home/아이디

읽기 + 쓰기

이 과목 작업 공간이자 제출함입니다.

산출물은 week01, week02 처럼 주차별 폴더로 정리합니다. 오늘은 week02.

다른 학생은 내 폴더를 볼 수 없습니다.

```
$ ls ~  
  README.md  내키이름.txt  
  
# 내 홈의 목록. 내키이름.txt 는 등록한 API 키 파일
```

공용 배포 폴더 /home/aisw25

읽기 전용

수업 자료가 week01, week02 처럼 주차 폴더로 배포됩니다.

해당 폴더를 내 폴더로 복사해 온 뒤, 내 폴더에서 작업합니다.

```
$ cd ~  
$ cp -rn /home/aisw25/week02 ~/  
$ ls week02  
강의안_02주차.md  내소재_양식.md  맛집  
슬라이드_02주차.pdf  운동  채용  포트폴리오  
# 맛집·운동·채용·포트폴리오가 오늘의 예제 폴더  
  
$ cp -n week02/내소재_양식.md week02/내소재.md  
  
# 방금 만든 week02/내소재.md(양식의 사본)를 VS Code에서 열어 띄워 둡니다.  
오늘 답은 여기에
```

파일 수정 작업은 반드시 배포 폴더 파일 복사 → “내 폴더”에서 합니다.

내 PC로 내려받기: 슬라이드 PDF와 강의안 (열람용)

1. 새 PowerShell 창을 하나 더 엽니다 (내 로컬 PC)

서버 접속 창은 그대로 둡니다. 앞 장의 cp는 서버 안 복사, scp는 내 PC로 전송입니다.

2. scp 명령으로 PDF와 .md를 내 PC로 복사

아이디와 비밀번호로 접속합니다. 명령 끝의 마침표(.)는 "지금 이 폴더로"라는 뜻이며, 프롬프트(PS C:\Users\이름>) 뒤의 명령만 입력합니다.

서버 창([아이디@aisw02 ~]\$)에서 치면 내 PC가 아니라 서버 안에 내려받아집니다.

3. 슬라이드 PDF를 열어 놓고 시작합니다

start 명령을 치면 바로 열립니다. 자기 화면에 띄워 놓고 따라옵니다.

4. 강의안 .md는 VS Code로 읽습니다

파일을 열고 Ctrl+Shift+V(미리보기) · Ctrl+K V(나란히 보기).

[내 PC · PowerShell] 새 창 · 서버 접속 창은 그대로

```
PS C:\Users\이름> scp 아이디@aisw02.tedso.kr:/home/aisw25/week02/*.pdf .
아이디@aisw02.tedso.kr's password:
슬라이드_02주차.pdf          100%
PS C:\Users\이름> scp 아이디@aisw02.tedso.kr:/home/aisw25/week02/강의안
_02주차.md .
강의안_02주차.md          100%
PS C:\Users\이름> start 슬라이드_02주차.pdf

# PDF가 아직 없으면 강의안만 받고 먼저 봅니다
```

확인 포인트 내 PC에 슬라이드 PDF와 강의안 .md가 받아졌고, PDF가 화면에 열렸다.

지난주 세 부분

1

바이브 코딩 체험

말로만 만들기

Artifacts · Design · Gemini · Stitch에
웹페이지를 시켰습니다. 결과는 시킬때마다
다르고 내부 구조는 아무도 모릅니다

실제 사고 4건

개인정보 유출 · 운영 DB 삭제 ·
배포 이틀 만에 공격 · 앱 170개 노출
공통점: 요구사항·검증·보안 설계 없음

강의 시장의 착각

딸깍 수익화 환상, 현업 눈높이 부재

2

방향과 규칙

바이브 → 에이전틱 코딩

요구사항 문서를 폴더에 두고,
결과는 테스트 코드로 검증합니다.
돌다리 두드리기는 AI에게 맡깁니다

로드맵 4단계

프롬프트 → 컨텍스트 → 하네스 → 루프

다섯 역량 · 규칙 4

풀스택·데이터·아키텍처·안전·운영
AI 결과를 설명할 수 있을 것,
결과물은 산출물·검증 코드·의도 기록

3

첫 프로그램, 두 방식

① 말로만: "숫자 게임 만들어줘"

돌아가긴 하지만 힌트 문구, 잘못된
입력 처리 같은 세부는 AI가 정했습니다

② 요구사항 → 테스트 → 통과

REQUIREMENTS.md →

test_guess.py

→ guess.py, pytest 통과로 증명

차이는 "문서"와 "테스트"

문서와 테스트가 남으면 다음 수정에서
깨진 곳을 자동으로 잡아냅니다

지난주 세 질문과 에이전틱 코딩 방법

1

① AI가 스스로 정한 것

질문

내가 말하지 않았는데
AI가 스스로 정한 것은 무엇인가?

답

요구사항에 정의되지 않은 항목은
AI의 임의 판단으로 채워집니다.
요구사항 명세를 먼저 작성해
이 누락을 제거합니다.

= SDD · 명세 주도 개발

2

② 맞는지 확인할 방법

질문

이 코드가 요구대로 동작하는지
어떻게 확인하는가?

답

요구사항마다 기대 결과를 테스트 코드로
작성해 자동으로 검증합니다.
사람의 눈이 아니라 테스트 통과 여부가 정
확성의 기준이 됩니다.

= TDD · 테스트 주도 개발

3

③ 요구를 더 붙일 자리

질문

요구사항을 하나 추가하려면
무엇을 수정해야 하는가?

답

코드가 아니라 요구사항 명세서에
요구사항을 추가하고, 그에 맞춰 테스트와
구현을 갱신합니다.
명세가 변경의 단일 기준입니다.

= SDD · 명세 주도 개발

지난주 체크리스트

- ✓ 개인 폴더에 README.md 가 있다 (지난주 제출물)
- ✓ week01 폴더를 받았고 그 안에 AI가 만든 첫 프로그램이 있다
- ✓ 같은 게임을 요구사항 → 테스트 → 통과로 다시 만들었다
- ✓ REQUIREMENTS.md · test_guess.py · guess.py 세 파일이 있고 테스트가 통과했다
- ✓ (선택) 내 소재로 3종 프롬프트를 AI에게 만들게 해 봤다
- ✓ 세 가지 질문(AI가 정한 것 · 확인 방법 · 요구 추가 자리)을 생각해 봤다
- ✓ 한 학기 규칙 네 가지를 들었다 (공식 문서 · AI 팀원 · 설명 가능 · 3종 세트)

*# 없어도 오늘 실습에는 지장이 없습니다. 앞의 접속 · SERVER MAP · DOWN
LOAD 장만 마치면 됩니다*

WHY

이번 주는 왜 명세인가

지난주 세 질문 중 둘(①·③)의 답이 요구사항 명세를 가리켰고, 그 방법이 SDD(명세 주도 개발)였습니다.

SDD는 명세를 먼저 확정하고 그 명세가 구현과 검증을 주도하게 하는 방식입니다.

이번 주는 이 사이클의 첫 단계인 명세를, 다음 주는 검증(TDD)을 다룹니다.

SDD의 첫 단계 구현 앞에 명세

고민

지난주에는 "만들어줘" 한 줄에서 바로 구현으로 갔고, 힌트 문구 같은 세부는 AI가 정했습니다.

이번 주의 답

명세를 먼저 쓰고, 그 명세가 구현과 검증을 이끅니다. 명세에 없는 것은 빈틈으로 드러나 사람이 정합니다.

문서가 없는 코드 분석

고민

문서가 없거나 사라진 시스템, 인수인계받은 코드, 관심 있는 상용 서비스. 분석을 시작할 명세가 없습니다.

이번 주의 답

완성물에서 명세를 뽑아내는 리버스 엔지니어링을 배웁니다. SDD를 역방향으로 쓰는 것입니다.

말이 어긋나는 지시 명세 문장

고민

"잘 되게 해줘"라고 시키면 AI는 빈 정보를 그럴싸한 추측으로 채웁니다.

이번 주의 답

조건과 행위가 든 **EARS 문형으로, 사람과 AI가 같은 뜻으로 읽는 명세 문장**을 쓰는 법을 배웁니다.

이번 주 완성물(코드 · 화면) → 명세서 → 읽고 고치기 → 재생성. 재생성본이 원본과 달라진 부분이 명세의 빈틈입니다.

이번 주 학습 목표

1

완성물 → 명세서 복원

문서가 없거나 사라진 시스템, 관심 있는 상용 서비스를 분석해 요구사항 명세서로 재구성합니다. 코드에 감춰진 지식을 조직의 자산으로 꺼냅니다.

인수인계 · 레거시 현대화 · 경쟁 서비스 분석

2

누구나 동일하게 읽는 명세

“로그인이 잘 되게 한다”를 “잘못된 비밀번호를 입력하면, 오류 메시지를 표시해야 한다”로 교정해 씁니다.

기획 · 개발 · 테스트 · AI의 공통 계약

3

완료 조건 & 실행 전 통제

지시마다 완료 조건을 붙이고, Plan 모드로 계획을 먼저 받아 실행 전에 검토합니다.

*일을 검증 가능한 단위로 분해하는 것은 다음 주(TDD)에서 이어집니다.

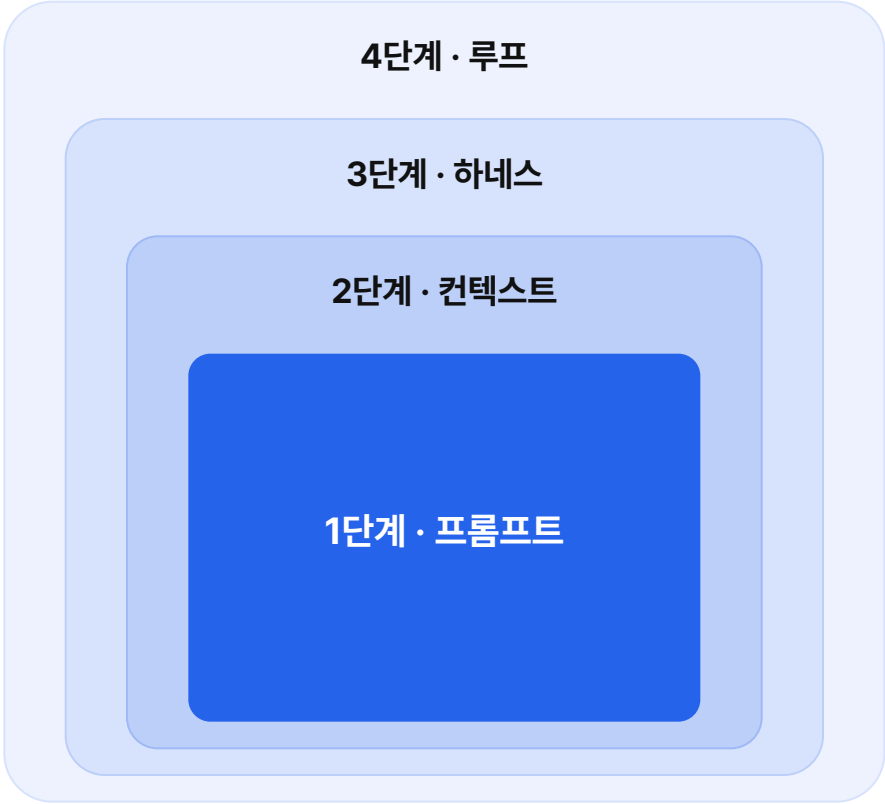
AI든 동료든 통하는 “위임”의 원리

이 과목의 방향

과목의 기반: AI 활용법 진화 로드맵

도구는 빠르게 바뀌지만 네 단계의 구분은 대개 그대로 쓰입니다. 지시에서 그치지 않고 검증까지 시키는 것이 이 과목의 목표입니다.

각 단계는 앞 단계를 포함하며 확장



AI가 만들고, 사람이 검증한다. 각 단계는 해당 주차에서 다룹니다.

1단계 · 프롬프트 엔지니어링

AI에게 무엇을 할지 매번 정확하게 지시하는 법. 모든 활용의 출발점입니다.

2단계 · 컨텍스트 엔지니어링

AI에게 무엇을 보여 주고 일을 시킬 것인가.
올바른 판단을 하도록 전체 정보를 갖춰 줍니다. 대표 도구는 CLAUDE.md.

3단계 · 하네스 엔지니어링

강제 검증 장치를 붙여 결과를 믿을 수 있게 만드는 틀. MCP(외부 도구 연결 규약) · Skill(재사용 절차 문서) · Hook(정해진 시점에 자동 실행되는 스크립트)으로 답변의 안정성을 보장합니다.

4단계 · 루프 엔지니어링

일을 맡기면 계획·실행·검증을 스스로 도는 단계.
완성될 때까지 피드백과 개선을 반복합니다 (에이전트 · 자동화).

로드맵 네 단계, 각각에서 갈리는 것

앞 장의 네 단계를 같은 일(게시판 만들기)에 적용한 것입니다.

왼쪽은 주변에서 흔히 보는 방식으로, 단계마다 무엇이 빠져 있는지가 오른쪽과의 차이입니다.

아무렇게나 시킬 때 · 규칙 없이

① 프롬프트 · "게시판 만들어 줘"

완료 조건 없이 한 줄로 시킵니다. 권한·오류 처리는 AI가 알아서 정하고, 무엇이 만들어졌는지 시킨 사람도 모릅니다.

② 컨텍스트 · 세션마다 처음부터 다시 설명

새 대화마다 프로젝트 배경과 규칙을 말로 다시 알려 줍니다. 빠뜨린 날은 다른 결과가 나오고, 팀원마다 다르게 시켜 결과가 갈립니다.

③ 하네스 · 무엇이든 허용

AI가 하겠다는 것을 모두 Yes로 넘깁니다. 파일 삭제와 운영 DB 접근이 막히지 않아, 지난주 Replit처럼 사고가 나고서야 알게 됩니다.

④ 루프 · 결과를 채팅으로 계속 고치기

결과를 눈으로 보고 "여기 고쳐 줘"를 반복합니다. 고친 곳이 다른 곳을 깨뜨려도 알 수 없고, 언제 끝나는지도 정할 수 없습니다.

제대로 시킬 때

① 프롬프트 · 완료 조건을 붙인 지시

"비회원 작성 시 로그인 화면 이동 · pytest 3건 통과. 수정할 파일 목록을 먼저 보여라."

② 컨텍스트 · CLAUDE.md에 규칙을 적어 둔다

세션마다 자동으로 읽혀 매번 적용됩니다.

③ 하네스 · 권한과 자동 차단

"테스트 없이 커밋 금지"는 흑으로 차단, 운영 권한은 처음부터 주지 않습니다.

④ 루프 · 반복 자체를 설계

"실패하는 테스트를 모두 통과할 때까지 수정과 실행을 반복하라. 10회를 넘기면 중단하고 보고하라."

새 기능이 나오면 대개 이 네 단계 중 하나를 더 편하고 정확하게 쓰게 하려고 나옵니다. 어느 단계를 강화하는지 보면 활용 방법을 알게 됩니다.

새 기능은 로드맵의 단계를 강화한다

새 기능은 사람이 AI를 더 편하고 정확하게 쓰도록, 각 단계를 강화할 목적으로 나옵니다.

MCP: 도구 연결

Model Context Protocol. AI가 외부 도구·데이터(GitHub, DB, 브라우저, 사내 시스템)를 같은 방식으로 부르게 하는 공개 규약. 도구마다 따로 붙이던 연결을 표준 하나로 통일합니다.

한 번 만든 MCP 서버는

Claude·Cursor·ChatGPT 어디서든 그대로 쓰여, AI가 문서 검색을 넘어 실제 시스템을 조회하고 조작하는 일까지 맡을 수 있습니다.

[공식 문서](#) · modelcontextprotocol.io

로드맵: 하네스, 컨텍스트

③ 하네스: AI가 쓸 수 있는 도구와 그 범위를 정합니다. ② 컨텍스트: MCP로 가져온 자료가 AI의 입력이 됩니다.

메모리: 세션 너머의 기억

대화가 끝나도 남는 정보. 사람이 적어 두는 CLAUDE.md와, AI가 작업 중 배운 것을 스스로 파일에 적어 다음 세션에 읽는 자동 메모리 두 종류가 있습니다.

<https://code.claude.com/docs/en/memory>

기업용 서비스도 같은 기능을 냅니다. 국내 [웍스AI](#)는 2026년 7월 회사 전체에 메모리를 한 번에 켜는 옵션을 냈습니다.

로드맵: 컨텍스트

② 컨텍스트. "세션마다 다시 설명"을 없앱니다. 무엇을 기억할지는 사람이 정합니다.

LLM 위키: 압축된 지식 저장소

Karpathy가 2026년 4월 제안한 방식. 원본 자료(raw) · AI가 정리한 위키(wiki) · 운영 규칙(schema) 세 폴더로 이루어집니다.

RAG처럼 질문마다 원문을 다시 검색하는 대신, AI가 자료를 넣을 때마다 개념·인물별 마크다운 페이지를 쓰고 서로 링크하며 갱신합니다. 지식이 쌓이고, 모순과 빈틈이 페이지에 드러나며, 사람이 읽고 고칠 수 있습니다.

[원문](https://gist.github.com/karpathy) · gist.github.com/karpathy

로드맵: 컨텍스트, 하네스, 루프

② 컨텍스트가 중심. AI가 위키를 스스로 관리하는 부분은 ③ 하네스(스킬)와 ④ 루프에 걸칩니다.

경계에 걸치는 기능 MCP는 연결(하네스)이면서 자료(컨텍스트)를 나르는 여러 기능이 있습니다. 도구 자체의 발전은 어느 단계에 속한다고 단정하기 어렵습니다.

CLAUDE 업데이트 체인지로그와 AI 로드맵

공식 체인지로그 최근 항목을 보며 "이건 어느 단계인가"를 생각합니다.

<https://code.claude.com/docs/en/changelog>

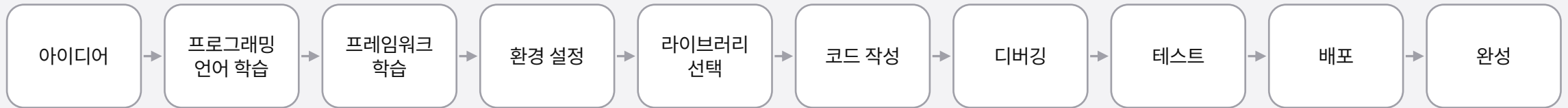
- ① /status에 조직 정책을 못 읽은 이유 표시 → 조직 정책은 회사가 모든 세션에 강제하는 규칙이니 하네스
- ② 텔레메트리(원격 제어)에 user.email/user.groups 추가 → 하네스
- ③ --plugin-dir로 폴더 단위 플러그인 로드·하트리로드 지원 → 하네스
- ④ 도구 결과 디스크 저장 1GB 상한 + 잘림 표시 → 컨텍스트 (도구 결과가 대화에 들어가는 양 제어)
- ⑤ /rewind 되감기 → 컨텍스트
- ⑥ /model opusplan[1m]이 "Model not found"로 거부되던 문제 수정 → 하네스 (모델 선택 명령)
- ⑦ 새 모델 기본 적용 → 해당 없음(성능)

왜 문서가 먼저인가

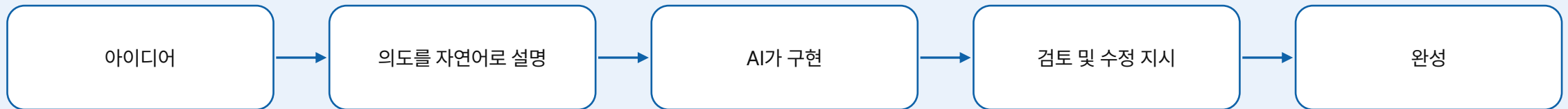
개발 과정의 변화

지난주 바이브 코딩은 아래 그림입니다. 과거 개발 과정 단계 몇 개가 현재 개발 과정(바이브 코딩)에 보이지 않습니다.

과거의 개발 과정



현재의 개발 과정 (바이브 코딩)



환경 설정·디버깅·테스트·배포 단계는 어디로 간 걸까요?

사고: 개발자를 줄였다가 다시 고용

Gizmodo · 404 Media 보도

After AI Led to Layoffs, Coders Are Being Hired to Fix 'Vibe-Coded' Screwups

사고

AI만 믿고 개발자를 줄인 회사와, 개발자 없이 제품을 만든 PM·영업·소상공인이 완성 직전에 막혔습니다. 기능 하나를 더 붙이면 다른 곳이 깨지고, AI 사용료만 늘어 처음부터 다시 만드는 편이 싼 상태가 됐습니다.

수습

경력 개발자를 다시 고용했습니다. Fiverr의 수리 전문가 Hamid Siddiqi는 15~20개 고객사와 상시 작업 중이고, VibeCodeFixers.com에는 경력 개발자 300명 이상이 등록했습니다. 흔한 의뢰: 정리되지 않은 UI, 성능을 떨어뜨리는 코드, 돌아가지만 어설픈 기능, 인증 구멍.

<https://www.404media.co/the-software-engineers-paid-to-fix-vibe-coded-messes/>

착각과 실제

착각

"AI가 코드를 바로 만들어 주니 개발 과정도, 개발자도 필요 없다."

실제로 건너뛴 단계

요구사항 명세 · 설계 · 테스트. 머릿속 생각을 그대로 던져 구현하니 무엇을 만들었는지 만든 사람도 모르고, 고칠 때마다 다른 곳이 깨집니다.

누가 메웠나

건너뛴 단계를 아는 사람, 즉 경력 개발자입니다. 뒤늦게 명세를 복원하고 테스트를 붙이는 일이 "바이브 코드 수리"라는 직업이 되었습니다.

<https://vibecodefixers.com/>

비용은 어디로 갔나

건너뛴 단계의 비용은 수리비와 재작업으로 돌아왔습니다.

사고: 상담을 AI로 대체했던 Klarna

Bloomberg · TechCrunch 보도

Klarna CEO says company will use humans to offer VIP customer service

사고

2024년 2월 AI 상담원이 첫 달 230만 건, 700명분 일을 한다고 발표하고 채용을 멈췄습니다. 인력 5,500 → 약 3,000명. 분쟁·복잡한 환불에서 품질이 떨어지자 CEO는 "비용에 너무 집중해 품질이 낮아졌다"고 인정했습니다.

수습

2025년 5월 사람 상담원 채용을 재개. 외주하던 수천 명분을 학생·지방 거주자 중심 원격 인력으로 다시 채우고, AI는 단순 문의, 사람은 어려운 문의를 맡는 혼합 구조로 바꿨습니다.

<https://techcrunch.com/2025/06/04/klarna-ceo-says-company-will-use-humans-to-offer-vip-customer-service/>

개발만의 일이 아니다

상담에도 과정과 전문성이 있었다

고객의 문제를 파악하고 해결까지 끌고 가는 그 직군의 과정입니다. 경영진은 그 과정을 AI가 통째로 대신할 수 있다고 봤습니다.

실제로 건너뛴 것

"어떤 문의를 AI가, 어떤 문의를 사람이 맡는가"의 구분. 평균 처리 시간과 만족도만 보다가, 어려운 문의가 반복 접촉으로 돌아오는 것을 놓쳤습니다.

누가 메웠나

그 과정을 아는 사람입니다. 사고를 낸 쪽과 수습한 쪽이 다릅니다. 개발과 같은 구조입니다.

남은 것

AI는 여전히 문의의 2/3를 처리합니다. 없어진 것은 "사람이 필요 없다"는 판단이고, 남은 것은 역할을 나눈 설계입니다.

잘 되는 조건 (Karpathy, 2026)

X 게시물

Andrej Karpathy · 전 Tesla AI 총괄 · OpenAI 창립 멤버

무슨 일이 있었나

"코딩 에이전트는 12월 전에는 사실상 안 됐고, 그 뒤로는 사실상 된다." 30분 동안 서버 설정 · 모델 벤치 · 웹 UI · 서비스 등록 · 보고서까지 에이전트가 혼자 끝낸 주말 프로젝트를 공개했습니다.

잘 된 조건

"잘 명세된 일, 기능을 검증·테스트할 수 있는 일에서 훨씬 잘 된다."

"핵심은 일을 알맞게 분해하는 감각이다. 되는 부분을 넘기고 가장자리를 사람이 돕는다."

"높은 수준의 방향, 판단, 감각, 감독, 반복이 필요하다."

<https://x.com/karpathy/status/2026731645169185220>

사고의 반대편, 조건은 같다

같은 도구, 다른 결과

앞 두 사례가 건너뛴 단계(명세 · 검증 · 역할 분담)를 Karpathy는 사람이 미리 채웠습니다.

빈 단계를 누가 메우는가

앞 두 사례는 사고가 난 뒤 경력자를 고용해 메웠고, 여기서는 시키는 사람이 처음부터 메웠습니다. 단계는 **없어지지 않았**습니다. 차이는 **미리 하느냐 나중에 비용을 내고 하느냐**입니다.

잘 되는 조건 = 이 과목의 순서

명세(잘 명세된 일) → 검증(테스트할 수 있는 일) → 분해(알맞게 나눈 일).
오늘 실습이 첫 번째 조건입니다. "에이전트를 병렬로 관리한다"는 말은 로드 맵의 루프 단계이고, 그 전에 프롬프트·컨텍스트·하네스가 있습니다.

"잘 명세되고 검증할 수 있는 일에서 잘 된다." → 명세 추출과 EARS

왜 문서가 먼저인가

결과부터 보는 바이브 → 의도부터 정하는 에이전틱



Andrej Karpathy ✓

@karpathy

...

A lot of people quote tweeted this as 1 year anniversary of vibe coding. Some retrospective -

I've had a Twitter account for 17 years now (omg) and I still can't predict my tweet engagement basically at all. This was a shower of thoughts throwaway tweet that I just fired off without thinking but somehow it minted a fitting name at the right moment for something that a lot of people were feeling at the same time, so here we are: vibe coding is now mentioned on my Wikipedia as a major memetic "contribution" and even its article is longer. lol

The one thing I'd add is that at the time, LLM capability was low enough that you'd mostly use vibe coding for fun throwaway projects, demos and explorations. It was good fun and it almost worked. Today (1 year later), programming via LLM agents is increasingly becoming a default workflow for professionals, except with more oversight and scrutiny. The goal is to claim the leverage from the use of agents but without any compromise on the quality of the software. Many people have tried to come up with a better name for this to differentiate it from vibe coding, personally my current favorite "agentic engineering":

- "agentic" because the new default is that you are not writing the code directly 99% of the time, you are orchestrating agents who do and acting as oversight.
- "engineering" to emphasize that there is an art & science and expertise to it. It's something you can learn and become better at, with its own depth of a different kind.

In 2026, we're likely to see continued improvements on both the model layer and the new agent layer. I feel excited about the product of the two and another year of progress.



Andrej Karpathy ✓ @karpathy · 2025년 2월 2일

There's a new kind of coding I call "vibe coding", where you fully give in to the

<https://x.com/karpathy/status/2019137879310836075>

일을 알맞게 분해하는 감각: AI가 할 일 vs 사람이 할 일

- 1 '분통' 터뜨린 고객들, '곡소리' 낸 상담사들
- 2 [단독] AI에 수천억 썼지만... "상담 만족도 20% 그쳐"
- 3 [단독] '경제허리' 30·40대 절반 "AI상담 불편해 경쟁사 옮길것"
- 4 [단독] 연령대 불문 "진보된 AI보다 '빠른 인간상담사 연결' 원해"
- 5 [단독] 카드사 콜센터 매년 500명 감소... 고객은 진 빠지고, 상담사는 해고 '위기'
- 6 **"AI 뽐뽐이에 고객은 이미 화 나있다... 적용방식 바꿔야"**
- 7 [체험기] 20대 기자도 앱서 상담원 연결 버튼 못찾고 찢찢
- 8 [인터뷰] "피할 수 없는 미래라는 AI... 고객도, 상담사도 과실못누려"
- 9 고객편의 AI의 역설... 은행권 콜센터 순호감도 '역성장'
- 10 [인터뷰] "AI, 상담사 대체 아닌 빠른 고객 문제해결 집중해야"

美 911 전화 AI가 받는다... 비응급 생활민원 대응 확산

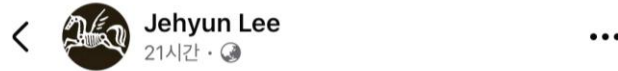
<https://www.digitaltoday.co.kr/news/articleView.html?idxno=588012>

한 사고에 신고 수십 통...미국 911이 AI를 투입한 이유 [담 넘어 시시티]

<https://www.citytimes.co.kr/news/articleView.html?idxno=90024>

왜 문서가 먼저인가

기획과 검증은 여전히 전문가의 일



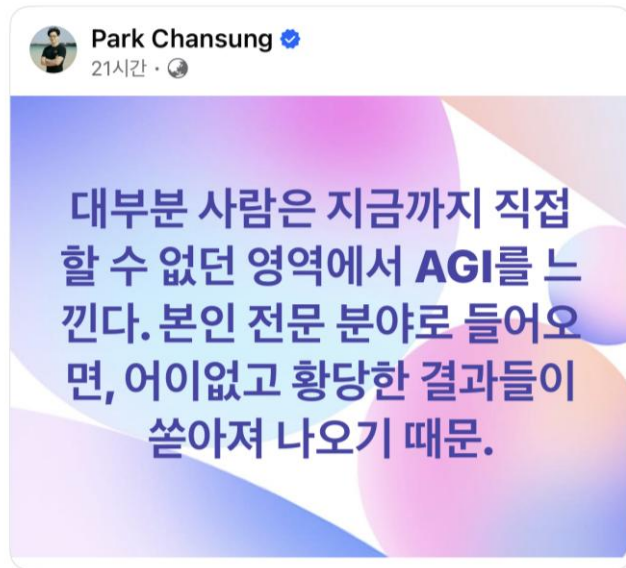
True.

AI 성능에 대한 논평은 두 갈래.

1. 자기가 잘 모르는 분야에 대해서는 찬양 일색이고,
2. 자기가 잘 아는 분야에 대해서는 아직 멀었다는 반응이 많다.

잘 아는 만큼 미진한 점이 잘 보일 것이고,
잘 아는 만큼 프로 인간과의 격차가 보이기 때문에.

ppt 딸깍월천을 광고하시는 분들 중에서 평소에 ppt 만드시던
분들이 얼마나 될지 진심으로 궁금하다.



68 10 1



왜 문서가 먼저인가

단계는 삭제되지 않고 압축된다

직군이 하던 일을 한 명이

기획자·디자이너·분석가·설계자·개발자가 나눠 하던 일을, 이제 한 명이 AI를 시켜 완수합니다.

팀의 모양도 바뀌었다

예전에는 기획·디자인·개발이 부서로 나뉘어 문서를 주고받았습니다. 지금은 한 팀에 앉아 함께 만들거나, 한 사람이 AI로 여러 역할을 다 합니다. 내 직군 옆의 일이 무엇인지 알아야 시킬 수 있습니다.

단계 자체는 그대로

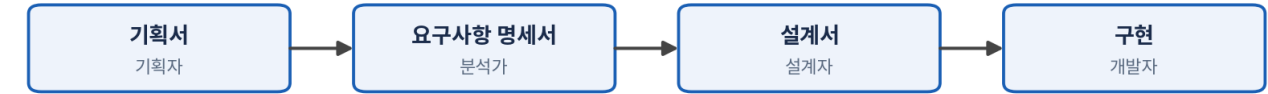
작업이 압축/통합되었을 뿐입니다. 삭제한 단계는 이후 비용으로 돌아옵니다.

각 단계의 산출물은 모두 문서

기획서, 요구사항 명세서, 화면 설계, 설계서. AI 협업에서는 이 문서들이 곧 AI의 컨텍스트가 됩니다.

그림 1. 개발 프로세스: 단계는 없어지지 않고 압축된다

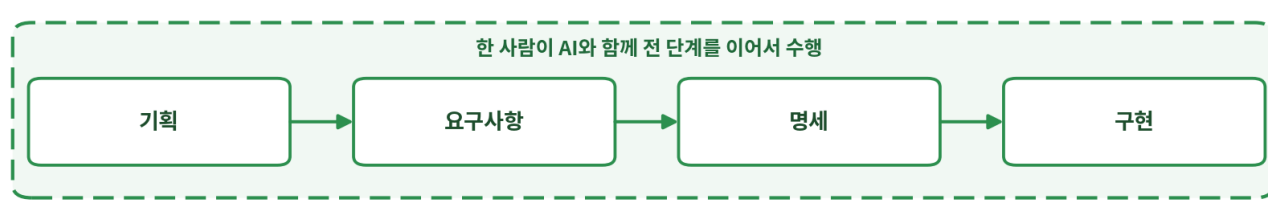
전통 개발: 단계마다 담당 직군이 있다



AI 시대의 유혹: 단계를 통째로 건너뛴다



이 과목의 방식: 단계는 유지하고, 수행을 압축한다



달라진 것은 단계의 소멸이 아니라 압축입니다. 직군이 나눠 맡던 단계를 한 사람이 AI와 함께 이어서 수행할 뿐, 단계 자체는 그대로 남아 있습니다. 건너뛴 단계는 반드시 문제로 돌아옵니다.

“환경 설정·디버깅·테스트·배포 단계는 어디로 간 걸까요?”

답: 빠진 상자는 「검토 및 수정 지시」에 압축돼 있습니다. AI는 정보가 비면 그럴싸한 답으로 채우고, 빈틈 없는 문서가 그 자리를 없앱니다.

왜 문서가 먼저인가

명세 주도 개발 (SDD): 코드가 아니라 의도가 기준이다

명세(spec)를 먼저 확정하고, 그 명세가 계획 · 구현 · 검증을 이끌게 하는 방식입니다.

GitHub는 이를 "코드가 진실의 원천"에서 "의도가 진실의 원천"으로의 이동이라고 설명합니다. 문서들이 곧 그 의도입니다.

원리

1. 고정된 '무엇'과 유연한 '어떻게'를 분리

명세는 무엇을 만들지, 코드는 어떻게 만들지입니다. **명세가 고정되면 코드는 언제든지 다시 만들 수 있습니다.** 사라진 줄 알았던 단계는 문서로 남깁니다.

2. 명세의 품질이 제품 품질의 상한

AI는 명세에 있는 것은 정확히, 없는 것은 추측으로 만듭니다. 기획·디자인·개발·테스트·AI가 **같은 문장을 읽어야 결과가 같아집니다.**

3. 변경은 명세에서 시작한다

기능을 바꾸려면 코드가 아니라 명세를 고치고 다시 생성합니다. **명세가 변경의 단일 기준입니다.**

흐름과 도구

SDD의 기본 흐름

요구 → 명세(spec) → 계획(plan) → 작업 분해(tasks) → 구현 → 검증.
GitHub Spec Kit과 AWS Kiro가 이 흐름을 명령으로 제공하고, Kiro는 명세를 EARS 문형으로 씁니다.

역방향도 같은 원리

완성물에서 명세를 뽑아내는 것이 오늘 실습입니다. 명세만 있으면 다시 만들 수 있다는 원리 1을 반대 방향으로 확인하는 것입니다.

참고

<https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>

<https://github.com/github/spec-kit>

<https://kiro.dev/docs/specs/>

AI를 정해 둔 틀 안에서 움직이게 한다는 점에서, 명세는 로드맵의 하네스 단계와 맞닿아 있습니다.

SDD를 거꾸로: 완성물에서 명세를 꺼낸다

설계도에서 완성물을 만드는 것이 엔지니어링, 완성물에서 설계 정보를 추출하는 것이 리버스 엔지니어링(역공학)입니다.

“명세만 있으면 다시 만들 수 있다”가 참이라면, 뽑아낸 명세로 다시 만든 것은 원본과 같아야 합니다.

명세와 “맛집 레시피”

맛집 요리 → 레시피

완성된 요리를 먹어 보고 재료와 순서를 적어 내면 레시피입니다. 코드나 화면을 보고 요구사항을 적어 내는 것이 명세 추출입니다.

레시피 → 같은 요리

레시피만 보고 다시 만들어 원래 요리와 비교합니다. 맛이 다르면 레시피에 빠진 것이 있는 것입니다. 재생성본이 원본과 다른 곳이 명세의 빈틈입니다.

레시피를 고치면 다른 요리

재료를 바꾸면 새 버전이 나옵니다. 명세에 내 요구사항을 추가해 원하는 것으로 만듭니다.

실습 방식

완성물 → 분석 → 명세서 → 읽기

명세서는 네 예제 모두 AI가 씁니다. 근거만 다릅니다. A · B는 공개 안내 페이지의 URL, C는 브라우저 AI가 지도 앱 화면을 조작해 적은 관찰 기록, D는 git clone으로 받은 코드를 AI가 읽은 것입니다.

읽기가 이번 주의 목적

AI가 쓴 명세서를 읽어 질의에 답하고, AI가 임의로 정한 곳 두 군데를 직접 고칩니다. 명세를 뽑는 것보다 읽고 활용하는 데 시간을 씁니다.

명세서 → 재생성 → 비교 → 내가 필요한 것

명세서만 주고 다시 만들게 합니다. 원본과 달라진 부분이 명세의 빈틈이고, 내 요구사항을 추가해 필요한 제품으로 만듭니다.

루프는 명세가 없어도 돌기 때문에, 애매한 명세는 사고가 된다

지시 하나를 받으면 Claude Code는 아래 세 단계를 스스로 완료까지 반복합니다(12장 로드맵 ④ 루프).
명세를 주지 않아도 루프는 멈추지 않고, 도구가 추측한 기준으로 "완료"를 선언합니다.

1 정보 수집 Gather Context	파일을 검색하고 읽어 상황을 파악합니다. 폴더에 둔 spec/요구사항명세서.md가 여기서 읽힙니다. CLAUDE.md(12장 로드맵 ② 컨텍스트)도 같은 자리에서 읽힙니다.
2 행동 Take Action	코드를 수정하거나 명령을 실행합니다. 무엇을 만들지는 명세의 REQ가 정합니다. REQ는 요구사항 문장 하나 (REQ-01, REQ-02...)로, "사용자가 ~하면, 시스템은 ~해야 한다" 형식입니다.
3 검증 Verify Results	테스트 · 실행 결과로 정확성을 확인합니다. 미완료면 1로 돌아가 반복. 명세와 테스트가 없으면 도구가 스스로 만든 기준으로 통과시킵니다.
언제든 사용자 개입	어느 시점에든 Esc로 중단하고 방향을 바꿀 수 있습니다. 명세를 고쳐 다시 넣는 것도 사람의 몫입니다.
우리 역할 명세로 기준을 정한다	명세를 폴더에 두면 정보 수집이 그것을 읽고, 검증이 그것을 기준으로 삼습니다. 오늘의 명세는 두 파일: 기능을 정하는 문장 명세(요구사항명세서.md)와 배치를 정하는 화면 명세(.pen). 예제 A · B · D에서 문장 명세를, 마지막 절에서 화면 명세를 직접 씁니다.

공통 실습 구성

- ✓ 실습 예제마다 작업 폴더가 있습니다. 해당 폴더를 열어 진행하세요.
- ✓ 과제는 week02/내소재.md 파일을 VS code로 열어 작성하세요.

A. 수영 강좌 찾기 · URL

안내 페이지 URL 하나에서 명세를 뽑습니다.

B. 채용 공고 · URL

재료만 채용 공고 URL로 바꿉니다.

C. 근처 맛집 · 화면 관찰

URL로 읽히지 않는 지도 앱. 브라우저 AI가 화면을 보고 근거를 만듭니다.

D. 포트폴리오 · 코드

git clone으로 받은 코드에서 명세를 뽑습니다. 화면 명세(Pencil) 대상입니다.

목적은 명세를 읽고, 내가 쓸 프로그램이라면 어떻게 바꿀지 판단

AI가 뽑은 명세에서 개선하고 싶은 부분을 직접 고칩니다.

확인 포인트 네 예제의 폴더 이름과 공통 절차의 순서를 안다.

공통 절차 · 예제마다 같은 순서

[준비] 예제 폴더에서 시작한다

결과물이 이 폴더에 쌓인다.

[Plan 모드] 읽기만 한다

구조와 기능 목록을 낸다. 페이지에 없는 것은 '추측' 표시

[Plan 해제] 명세를 저장한다

spec/요구사항명세서.md. REQ마다 EARS 라벨

[읽기] 내 명세로 만든다

읽기 질의에 답하고 한 곳을 고친다 ← 고침

[재생성] 명세만 보고 만든다

rebuild/에 명세서만. 원본은 보지 않는다

[발전] 내 것으로 바꾼다

요구 한 줄 · 내 정보를 넣고 다시 만든다

네 예제 모두 같은 단계. 바뀌는 것은 근거(URL · 메모 · 코드)뿐

막힐 때 대처법

원칙: 오래 붙잡지 않습니다. 멈추고, 대상을 좁히거나 바꿉니다.

증상별 대처

분석이 길어진다

Esc로 중단하고 범위를 줄여 다시 요청합니다.

예: "서비스 전체 구조를 추정하라" → "첫 화면(강좌 목록)의 구조만 추정하라".

세션이 끊겼다

`claude --continue`로 직전 세션을 이어갑니다.

실행 명령을 모른다

ls로 파일 이름을 보고 python3 파일이름.

모르면 Claude Code에 "실행 명령을 알려줘".

git clone이 안 된다

예제 D에서만 생깁니다. 키트 "포트폴리오"폴더에 들어 있는 index.html을 original/폴더에 복사하고 그대로 진행합니다(D 준비 장 참고).

자주 쓰는 것

Esc

작업 중단. 작업 중 개입

claude --continue

직전 세션 이어가기

Shift+Tab

권한 모드 순환 (plan ↔ 일반). 승인 질문은 No, keep planning

폴더를 신뢰하겠느냐고 묻는다

새 폴더에서 claude를 처음 띄우면 묻습니다. Yes.

참조 명령어 카드의 "작업 중 개입"과 권한 모드 순환 그림을 함께 둡니다.

레퍼런스: 광주시스포츠헬스센터 수영 프로그램 안내 페이지

리버스 엔지니어링의 대상은 잘 만들어진 사례입니다. 광주시스포츠헬스센터 수영 프로그램 안내 페이지 하나를 분석의 근거로 씁니다. AI가 페이지를 읽고 명세를 씁니다.

화면 · 관찰

오른쪽 URL을 브라우저로 엽니다. 어떤 강좌 · 시간 · 요금이 보이는지 잠깐 훑어봅니다. 표가 둘(청소년/성인 · 어린이) 있습니다.

왜 URL부터 시작하는가

받을 것도 설치할 것도 없어 절차 자체에 집중할 수 있습니다.

*코드를 받는 git clone은 D에서 새로 배웁니다.

가져오는 것은 구조와 항목

문장 · 이미지를 그대로 쓰지 않습니다.

*지도 앱처럼 JavaScript로 그려지는 화면은 URL만으로 읽히지 않아, 예제 C에서 화면을 보는 AI로 근거를 만드는 법을 봅니다.

[서버 터미널] · 예제 A 폴더 · 레퍼런스 URL

```
$ cd ~/week02/운동
```

```
$ ls
```

강좌.csv

[레퍼런스 URL]

<https://www.gjcs.or.kr/sportsCentre/program/swim>

광주시스포츠헬스센터 · 수영 프로그램 안내 · 강좌 · 요일 · 시간 · 대상 · 요금

[강좌.csv · 키트에 포함 · 페이지의 표 35행을 그대로 옮김]

프로그램, 시간, 반명, 요일, 정원, 대상, 이용료, 비고

확인 포인트 URL을 브라우저로 열어 강좌표 둘을 봤다.

준비: 두 터미널을 예제 A 폴더로 옮긴다

이 예제에서 우리가 새로 만드는 파일은 없습니다. 명세와 프로그램은 AI가 이 폴더에 만들고, 우리는 그 파일을 열어 읽고 고칩니다. 그래서 예제 폴더에서 시작합니다.

1. [서버 터미널] 예제 A 폴더로

cd ~/week02/운동. pwd로 위치를 확인합니다. 두 터미널의 폴더는 따로 움직이므로 각각 cd 합니다.

2. 폴더에 있는 것

강좌.csv(페이지 표를 옮긴 데이터, 재생성 때 씀)와
../내소재.md(키트 양식 · 답과 메모를 적는 곳).

둘 다 키트에 들어 있습니다.

[서버 터미널] · 폴더 확인

```
$ cd ~/week02/운동
$ pwd
/home/아이디/week02/운동
$ ls
강좌.csv
```

[이 예제가 끝나면 폴더에 쌓이는 것]

운동/	
강좌.csv	← 키트
spec/요구사항명세서.md	← 4/8 AI 저장 · 5/8 고침 (선택)
spec/프롬프트.md	← 4/8 AI 저장
rebuild/요구사항명세서.md	← 6/8 복사 · 8/8 한 줄 추가 (선택)
rebuild/강좌.py	← 6/8 · 8/8 AI 생성

확인 포인트 두 터미널이 ~/week02/운동에 있고, ls에 강좌.csv가 보인다.

결과 검토 pwd가 ~/week02/운동이고 ls에 강좌.csv가 보이면 준비 완료. 전원 같은 URL입니다.

Plan 모드에서 구조와 기능을 뽑는다

Plan 모드는 읽기와 분석만 되고 파일 수정은 잠기는 상태입니다. 분석 단계에서 파일이 바뀌는 사고를 막습니다.

1. [Claude Code 터미널] claude → Shift+Tab 두 번

예제 A 폴더에서 실행합니다(결과물이 이 폴더에 쌓임). 하단 표시가 plan으로 바뀝니다. 첫 실행이면 폴더 신뢰 Yes.

2. 구조 추정 프롬프트

AI가 페이지를 가져와 읽습니다(WebFetch 허용을 물으면 Yes). "읽을 수 없다"고 하면 서버 터미널에서 페이지를 받아 둡니다(오른쪽 아래 폴백). 페이지에 없는 것에는 「추측」 표시가 붙어야 합니다. **대상을 강좌표**로 못 박았습니다. 「이 서비스」라고만 하면 사이트 전체를 추측으로 채웁니다.

3. 기능 목록 프롬프트

화면 기능과 내부 처리를 구분하고 기능마다 관련 화면을 붙입니다. 이 목록이 명세서의 뼈대입니다. "계획을 승인할까요?"가 뜨면 No, keep planning.

확인 포인트 화면 목록 · 이동 흐름 · 「추측」 표시가 든 구조 추정과 기능 목록이 화면에 있다.

[Claude Code 안] · Plan 모드 · 두 프롬프트 (전문 입력)

```
$ cd ~/week02/운동
$ claude
# Shift+Tab 두 번 → 하단에 plan 표시

> https://www.gjcs.or.kr/sportsCentre/program_swim 페이지의 수영 프로그램 강좌표를 대상으로 구조를 추정하라. 완료 조건: ① 강좌표에 있는 정보 항목의 목록 ② 이 표를 보는 사람이 하려는 일의 추정 목록 ③ 표만으로 알 수 없는 것의 목록. 로그인 · 챗봇 · 다른 메뉴처럼 강좌표 밖의 기능은 대상이 아니다. 표에 없는 내용은 추측이라고 표시하라.

> 이 서비스의 기능 목록을 만들어라. 완료 조건: 각 기능은 한 줄 설명과 관련 화면 이름을 포함하고, 화면 기능과 내부 처리 기능을 구분한다.
```

응답 예시 · 입력 아님

- 화면: 성인 강좌표 · 어린이 강좌표 · 흐름: 프로그램 → 반
- 알 수 없음: 접수 방법, 잔여 정원
- [화면 기능] F1 반 목록 · F2 시간대 찾기 · F3 이용료
- [내부] F4 대상별 분류 · F5 정원 초과(추측)

폴백 · "읽을 수 없다"고 하면 서버 터미널에서 curl -sL https://www.gjcs.or.kr/sportsCentre/program_swim > 페이지.html 로 받아 두고, 프롬프트의 URL 자리를 "페이지.html"로 바꿔 다시 보냅니다.

명세 저장 → EARS 라벨 → 읽기

명세는 파일이어야 AI가 읽고 우리가 고칠 수 있고, EARS 라벨이 붙어야 검증할 수 있는 문장이 됩니다. 저장은 파일을 만드는 일이므로 Plan을 해제합니다.

1. Shift+Tab으로 Plan 해제

하단의 plan 표시가 꺼집니다.

2. 두 프롬프트를 이어서 보낸다

명세를 파일로 저장하고 EARS 라벨을 붙입니다.

3. 읽고 이해한다

VS Code 탐색기에서 spec/요구사항명세서.md를 클릭해 열고 REQ를 위에서부터 읽습니다. Ctrl+Shift+V로 md 파일을 가독성있게 볼 수 있습니다.

4. 개선할 부분을 찾아본다

내가 쓸 프로그램이라면 어떻게 할지 생각하며 읽습니다. 고치고 싶은 문장이 있으면 직접 고치거나 AI의 도움을 얻어 고칩니다(선택). 없으면 그대로 둡니다. 읽고 판단한 것이 핵심입니다.

확인 포인트 spec/에 명세서 · 프롬프트 파일이 있고, 각 REQ 앞에 문형 라벨이 붙었다.

[Claude Code 안] · Plan 해제 후 · 두 프롬프트 (전문 입력)

- > 요구사항 명세서를 만들어라.
 - 근거: 위 기능 목록
 - 형식: 한 문장(조건 + 할 일) · 고유 ID · 관련 화면
 - 저장: spec/요구사항명세서.md
 - 다시 만들 프롬프트: spec/프롬프트.md
- > 각 문장 앞에 EARS 문형 라벨을 붙여 다시 저장하라.
 - 라벨: 상시 · 사건 · 상태 · 원치 않는 동작 · 선택
 - 안 맞는 문장은 고쳐서 라벨을 붙여라
 - 보고: 고친 문장 목록

[명세 읽기 · vs Code에서 · 고치는 것은 선택]

- VS Code 탐색기에서 spec/요구사항명세서.md 클릭 → Ctrl+Shift+V로 미리보기
- 읽기 질의의 답은 week02/내소재.md에 · 고칠 곳이 없으면 "없음"
- 고치고 싶은 문장은 고치고 끝에 " ← 고침", Ctrl+S (선택)
- 직접 고치기 어려우면 Claude Code에 「REQ-03을 ...로 고쳐라」로 시켜도 된다

결과 비교 REQ 개수 · 고친 문장 · 빈 문형은 사람마다 다릅니다.

EARS: 요구사항을 정해진 문형으로 쓴다

EARS란 · Easy Approach to Requirements Syntax

요구사항을 자유로운 문장 대신 정해진 문형(템플릿)으로만 쓰게 하는 표기 법입니다. 롤스로이스의 Alistair Mavin이 제트엔진 제어 요구사항을 쓰면서 만들었고(2009), 지금은 시스템 공학의 표준 기법입니다.

자유로운 자연어의 세 문제

모호성 "잘", "적절히", "필요시"처럼 확인할 수 없는 말

불완전성 정상 흐름만 적고 오류·잘못된 입력은 빠짐

일관성 결여 같은 문장을 사람마다 다르게 읽음

기본 구조 · 조건과 행위를 분리한다

WHEN [조건/사건], THE SYSTEM SHALL [동작]

[조건]일 때, 시스템은 [동작]해야 한다. 문형을 강제하면 '언제'와 '무엇을'이 분리되어, 빠진 조건이 문장 안에서 드러납니다.

EARS 문형 · [조건], 시스템은 ~해야 한다		
문형	영문 구조	한국어 예
① 상시 Ubiquitous · 항상	The system SHALL ...	시스템은 모든 로그인 시도를 기록해야 한다
② 사건 Event-Driven · 사건 발생	WHEN ..., the system SHALL ...	사용자가 제출을 누를 때, 시스템은 데이터를 저장해야 한다
③ 상태 State-Driven · 상태 지속 중	WHILE ..., the system SHALL ...	파일이 업로드되는 동안, 시스템은 진행률을 표시해야 한다
④ 원치 않는 동작 Unwanted · 오류·예외	IF ..., THEN the system SHALL ...	만약 로그인이 5회 연속 실패하면, 시스템은 계정을 10분간 잠가야 한다
⑤ 선택 Optional · 선택 요소	WHERE ..., the system SHALL ...	결제 모듈이 설치된 경우, 시스템은 결제 메뉴를 표시해야 한다
⑥ 복합 Complex · 조건 결합	WHEN ..., WHILE ..., the system SHALL ...	업로드 중 사용자가 취소를 누르면, 시스템은 받은 파일을 지워야 한다

출처 Mavin et al., "Easy Approach to Requirements Syntax (EARS)", IEEE RE'09 (2009) · alistairmavin.com/ears

참고 실습 프롬프트는 ①~⑤에만 라벨을 붙입니다. ⑥ 복합은 사건과 상태가 겹친 문장이라 따로 라벨을 두지 않고 두 조건을 모두 씁니다.

AI 시대에 EARS가 중요한 이유: 빈칸을 AI가 채우지 않게

내가 말하지 않은 자리를 AI는 추측으로 채웁니다. 사람 개발자는 되물어 오지만 AI는 그냥 만듭니다. EARS는 문장마다 조건과 행위를 요구하므로, 빠진 자리가 코드가 아니라 문장 안에서 먼저 드러납니다.

1. 모호성 제거: AI가 채울 자리가 줄어든다

"언제"와 "무엇을"이 분리되니 예외 상황과 빠진 요소가 보입니다.
"잘", "적절히"는 어느 문형에도 들어가지 않습니다.

2. 사람과 AI가 같은 기준으로 읽는다

기획·개발·QA가 한 해석을 하듯 AI도 같은 뜻으로 읽습니다.
프롬프트의 모호함이 그대로 결과물의 편차가 됩니다.

3. 명세 주도 개발(SDD) 도구가 EARS를 쓴다

AWS Kiro 등은 요구사항을 EARS 형식으로 자동 변환·검증합니다.
명세 → 설계 → 작업으로 가는 첫 관문이 이 문형입니다.

4. 문장이 그대로 테스트가 된다

"만약 5회 실패하면 10분 잠금"에는 조건과 기대 결과가 들어 있어, 이번 주에는 사람이 실행해 눈으로 확인합니다. 다음 주 TDD에서는 같은 문장을 테스트 함수 하나로 옮겨 기계가 확인하게 합니다.

확인 포인트 내 명세서에서 조건이 없는 문장을 하나 찾았다. 그 자리가 지난 주 AI가 스스로 정했던 자리입니다.

● ● ● 작성 예시 비교 · 로그인 (같은 요구, 두 문장)

✓ 모호한 자연어 · 지난주 방식

사용자가 로그인 폼에 올바른 정보를 입력하고 제출하면 세션을 생성하고 대시보드로 이동시켜야 한다.

→ AI가 정할 것: '올바른'의 기준? 틀리면? 대시보드 주소는?

✓ EARS 문형 · [사건]

WHEN the user submits the login form with valid credentials,
THE system SHALL create a session and redirect to /dashboard.

[사건] 사용자가 유효한 자격 증명으로 로그인 폼을 제출하면, 시스템은 세션을 만들고 /dashboard로 이동해야 한다.

빈칸이 보여서 덧붙인 한 줄 · [원치 않는 동작]

[원치 않는 동작] 만약 자격 증명이 틀리면, 시스템은 오류 문구를 표시하고 로그인 폼에 머물러야 한다.

참고 EARS는 문장을 잘 쓰는 기술이 아니라 빈칸을 찾는 기술입니다. 4/8의 라벨 붙이기에서 어느 문형에도 안 들어가는 문장이 그 빈칸입니다.

EARS: 지시를 검증 가능한 문장으로

바이브 지시 (지난주 방식)

- ① 상시 "로그 좀 잘 남겨 줘"
- ② 사건 "저장 잘 되게 해줘"
- ③ 상태 "업로드 중에 뭔가 보여줘"
- ④ 원치 않는 동작 "로그인 에러 안 나게 해줘"
- ⑤ 선택 기능 "결제도 되면 좋고"

조건도 행위도 없어, 빈칸을 AI가 추측으로 채웁니다.

같은 요청, 다른 결과물

바이브 쪽은 무엇이 만들어졌는지 만든 사람도 설명하지 못합니다. 명세 쪽은 문장마다 확인 방법이 따라옵니다. "5회 실패 → 10분 잠금"은 그대로 검증 문장이 됩니다. 내부의 차이는 재생성 비교(예제 A 7/8)에서 직접 봅니다.

출처 <https://alistairmavin.com/ears/>
각 문형의 정의와 예문을 볼 수 있습니다.

EARS 명세 지시 · 같은 요청

- ① 시스템은 모든 로그인 시도를 기록해야 한다.
- ② 사용자가 제출을 누를 때, 시스템은 데이터를 저장해야 한다.
- ③ 파일이 업로드되는 동안, 시스템은 진행률을 표시해야 한다.
- ④ 만약 로그인이 5회 연속 실패하면, 시스템은 계정을 10분간 잠가야 한다.
- ⑤ 결제 모듈이 설치된 경우, 시스템은 결제 메뉴를 표시해야 한다.

참고 지금은 각 문장에 조건과 행위가 들어 있지만 확인하면 충분합니다.
다음 주 TDD에서 이 문장들이 그대로 테스트가 됩니다.

명세 읽기: 질의 4개에 답하고, 고칠 곳이 있으면 고친다

여기가 이번 주의 목적입니다. **AI가 뽑은 명세를 한 번은 내 눈으로 읽습니다.**
고칠 곳이 없으면 없다고 적으면 됩니다.

1. 명세서와 페이지를 나란히 연다

VS Code 탐색기에서 운동/spec/요구사항명세서.md 클릭(Ctrl+Shift+V로 미리보기). 브라우저에는 강좌표 페이지. REQ를 위에서부터 한 번 읽습니다.

2. 질의 ①~③에 답한다

답은 내소재.md 「명세 읽기」 칸에 REQ 번호와 함께. 해당 없음이면 "없음"이라고 씁니다.

3. ④ 내 프로그램에 맞게 한 곳을 고친다

정답은 없습니다. 내가 쓸 프로그램이라면 이렇게 되면 좋겠다 싶은 REQ 하나를 골라 문장을 바꾸고, 끝에 "← 고침"을 붙여 Ctrl+S. 오른쪽 예처럼 작은 변경이면 충분합니다.

확인 포인트 질의 4개의 답이 내소재.md에 있고, 고칠 곳이 있었으면 명세서에 ← 고침이 있다.

● ● ● 명세 읽기 질의 · 답은 내소재.md 「명세 읽기」 칸에

- ① "잘" · "적절히"처럼 확인할 수 없는 말이 든 REQ가 있으면 → 고친 문장. 없으면 "없음"
- ② 잘못됐을 때 (없는 번호 입력 · csv 없음)의 REQ가 있는가 → 없으면 한 줄 써 넣기 (선택)
- ③ REQ 하나를 확인 문장 한 줄로 ("~하면 ~가 보여야 한다")
- ④ **내 프로그램에 적용하고 싶은 부분 한 곳** → 문장을 바꾸고 ← 고침

● ● ● ④ 고친 예 · **내가 쓸 프로그램이라면**

시스템은 반을 프로그램별로 표시해야 한다

→ 시스템은 반을 시간 순으로 표시해야 한다 ← 고침 (저녁 반 찾기)

사용자가 대상을 고르면, 시스템은 그 대상의 반을 표시해야 한다

→ 시스템은 시작할 때 성인 반만 표시해야 한다 ← 고침 (나는 성인)

시스템은 이용료를 주3회 · 주2회로 표시해야 한다

→ 시스템은 주2회 이용료만 표시해야 한다 ← 고침 (주2회만 다님)

참고 ④는 옳고 그름이 아니라 "내가 쓸 프로그램이라면 어떻게 할지 판단하는" 자리입니다. 바꾼 이유를 괄호로 한 줄 남기면 8/8에서 다시 씁니다.

명세만으로 다시 만들기: CLI 구현

1. rebuild 폴더에 명세서와 강좌.csv만 복사합니다

페이지 주소는 넣지 않습니다. 앞 장의 Claude Code는 그대로 둔 채, 터미널을 하나 더 열어 예제 A 폴더에서 시작합니다.

2. rebuild에서 claude 실행, 재생성 요청

새 폴더의 첫 실행이라 폴더 신뢰를 물으면 Yes. 하단에 plan 표시가 없는지 확인하고 오른쪽 프롬프트를 보냅니다. 파일을 만들겠다고 물으면 Yes.

3. 질문 목록을 보고, 실행합니다

질문은 AI가 보고하는 것이고 우리가 답하지 않습니다. 내소재.md 「예제 A」 칸에 적어 둡니다. 이것이 명세의 첫 빈틈 후보입니다.

서버 터미널에서 ls로 파일 이름을 보고 그 이름으로 실행합니다. 화면에 무엇이 나오는지 봅니다.

[새 Claude Code 터미널] · rebuild 준비 (셀)

```
$ cd ~/week02/운동
$ mkdir -p rebuild
$ cp spec/요구사항명세서.md 강좌.csv rebuild/
$ cd rebuild
$ claude
# claude가 뜨면 여기부터 아래 카드 = Claude Code 안
```

[Claude Code 안] · 재생성 요청 (전문 입력) → • 은 응답 예시

> 이 폴더의 요구사항명세서.md만을 근거로, python3 하나로 실행되는 명령줄 프로그램(CLI)을 구현하라. 데이터는 같은 폴더의 강좌.csv를 읽고, 강좌.csv로 할 수 있는 기능만 만든다. 명세에 없는 내용은 추측하지 말고 부족한 정보는 질문 목록으로 정리해 보고하되, 답을 기다리지 말고 명세에 있는 것만으로 구현하라

• 질문: 이용료의 성인 · 청소년 구분이 명세에 없습니다 / 정원이 찼을 때 무엇을 보여줍니까?

• rebuild/강좌.py 를 만들었습니다.

[서버 터미널]

\$ ls && python3 (그 이름)

확인 포인트 다시 만든 프로그램이 실행되고 질문 목록을 확인했다.

결과 비교 재생성의 목적은 복제가 아니라 빈틈 드러내기입니다. 같은 명세라도 결과는 사람마다 다릅니다.

만든 프로그램을 써 본다: 내 조건으로 찾을 수 있나

과제가 아니라 관찰입니다. 결과는 사람마다 다르니, 정답 화면을 맞히는 것이 아니라 내 결과가 어느 갈래인지 찾습니다.

1. 프로그램을 실행한다

ls로 이름을 보고 python3 (그 이름). 메뉴가 있으면 눌러 보고, 없으면 나온 대로 봅니다.

2. 페이지에서 하던 것과 비교한다

브라우저의 강좌표에서 나는 시간 · 요일 · 대상으로 훑어 골랐습니다. 프로그램에서 그것을 할 수 있습니다.

3. 오른쪽 표에서 내 결과에 해당하는 것을 찾는다

내소재.md 「예제 A · 빈틈」 칸에 한 줄. 8/8에서 명세에 더할 재료입니다.

[서버 터미널] · 실행 결과를 페이지와 나란히

```
$ cd ~/week02/운동/rebuild
$ ls
$ python3 강좌.py
# 파일 이름은 AI가 지어서 사람마다 다릅니다. ls로 본 이름으로 실행합니다
```

결과 분류 예시 · 오른쪽 칸이 8/8에서 명세에 더할 한 줄

프로그램 상태	명세에 없던 것	추가할 명세
조건으로 고르지 못하고 전체 목록만 나온다	「사건」 문형	[사건] 시간대를 고르면 그 시간대의 반만 표시해야 한다
없는 번호를 넣으면 오류로 멈춘다	「원치 않는 동작」 문형	[원치 않는 동작] 없는 번호를 넣으면 안내를 표시해야 한다
명세에 있는 항목이 프로그램에 없다	csv에 그 항목의 데이터	명세를 csv에 있는 항목으로 고쳐 쓴다
내 조건으로 찾을 수 있다	없음 (문형이 다 있었다)	더 있으면 좋을 기능 한 줄
결과가 표가 아니라 터미널 글자로 나온다	출력 매체 (AI가 터미널로 정함)	[상시] 결과를 브라우저에서 여는 HTML 표로 보여야 한다

확인 포인트 프로그램을 실행해 봤다. 내 결과가 어느 갈래인지 찾고, 내소재.md에 한 줄 적었다.

재생성물의 모양은 명세에 어떤 문형이 들어 있었는지가 정합니다. 4/8에서 붙인 라벨을 다시 보면 왜 그 모양인지 설명할 수 있습니다. 표 모양 · 색이 다른 것은 틀린 것이 아니라 명세가 정하지 않아 AI가 고른 것이고, 바꾸고 싶으면 그것도 명세에 씁니다.

발전: 명세에 한 줄을 더하고 다시 만든다

코드가 아니라 명세를 고치고, 프로그램은 명세에서 다시 나오게 합니다.
rebuild/에서 `claude --continue`.

1. 요구 한 줄을 명세에 직접 쓴다 (선택)

VS Code로 rebuild/요구사항명세서.md를 열고(spec/이 아닙니다) 마지막 줄 아래에 씁니다. 재료는 7/8의 빈틈, 없으면 오른쪽 예에서 하나를 고르거나 "내 프로그램이라면 더 있으면 좋을 것". 다음 REQ 번호 · 문형 라벨 · 끝에 " ← 추가", Ctrl+S.

2. 명세만 보고 다시 만들게 한다

오른쪽 프롬프트 하나. 요구는 프롬프트에 쓰지 않습니다.
추가한 문장이 명세에 있으니 프로그램은 거기서 나와야 합니다.

3. 다시 실행해 확인한다

ls로 이름을 보고 그 이름으로 실행합니다. 이제부터 A의 최신 명세서는 rebuild/요구사항명세서.md. spec/은 추출 당시의 기록입니다.

확인 포인트 다시 만든 프로그램이 실행된다. 한 줄을 추가했다면 그 요구가 반영되어 있다.

● ● ● 요구사항명세서.md · 추가 예시 · 하나 고르거나 참고해 씁니다

[사건] 사용자가 시간대(예: 19시 이후)를 고르면, 시스템은 그 시간대의 반만 보여야 한다. ← 추가

[원치 않는 동작] 만약 강좌.csv가 없으면, 시스템은 오류 대신 안내 문구를 표시해야 한다. ← 추가

[상태] 정원이 다 찬 동안, 시스템은 그 반에 "마감"을 표시해야 한다. ← 추가

[선택] 청소년이면, 시스템은 청소년 이용료를 표시해야 한다. ← 추가

[상시] 시스템은 결과를 브라우저에서 여는 HTML 표로 보여야 한다. ← 추가

● ● ● [Claude Code 안] · rebuild/ · 발전 프롬프트 (전문 입력)

```
$ cd ~/week02/운동/rebuild && claude --continue
```

```
> 요구사항명세서.md에 ← 추가 표시된 요구를 반영해 다시 구현하라. 명세에 없는 것은 추측하지 말고 질문하라. 변경한 파일과 반영한 REQ를 보고하라.
```

```
# [서버 터미널]
```

```
$ ls
```

```
$ python3 (그 이름)
```

내 것으로 내가 이 프로그램을 쓴다면 명세에 어떤 한 줄을 더하겠나. 실행은 안 해도 됩니다. 내 소재.md 「내 것으로」 칸에 한 줄.

명세를 뽑고, 읽고, 수정해서 다시 만들었다

공통 절차의 여섯 단계를 함께 진행했습니다.

[준비] 예제 폴더에서 시작 · 1~2/8

URL을 열어 보고, 두 터미널을 운동/ 폴더로. 만든 파일은 없음.

[Plan 모드] 읽기만 · 3/8

AI가 페이지를 읽고 구조와 기능 목록을 냈다.

[Plan 해제] 명세를 저장 · 4/8

spec/에 명세서 · 프롬프트 저장, REQ마다 EARS 라벨.

[읽기] 내 명세로 · 5/8

질의 4개에 답하고 한 곳을 손으로 ← 고침.

[재생성] 명세만 보고 · 6~7/8

rebuild/에 명세서 · csv만 두고 만들게 하고, 실행해 봤다.

[발전] 명세에 한 줄 · 8/8

← 추가 한 줄. 코드는 손대지 않았고 프로그램은 명세에서 다시 나왔다.

● ● ● 되짚어 볼 것 · 답은 내소재 .md 「생각 거리」 칸에

공개 페이지 하나로 명세 8~12줄이 나왔다.

명세에 없어서 프로그램이 하지 못한 일은?

고칠 곳이 없었다면 그대로 두어도 됩니다.

[폴더에 남은 것]

spec/요구사항명세서.md

spec/프롬프트.md

rebuild/요구사항명세서.md

rebuild/강좌.py

라벨 · ← 고침 한 곳

명세를 다시 만드는 프롬프트

← 추가 한 줄 · 최신 명세

명세에서 나온 프로그램

다음 예제 B 채용 공고. 근거만 다른 URL로 바뀌고 같은 여섯 단계를 진행합니다.

채용시험정보: 공고 목록 페이지

A와 같은 광주시의 페이지입니다. 광주시청 채용시험정보 게시판 한 쪽이 분석의 근거이고, 「공고 찾아보기」 서비스의 명세가 나옵니다.

1. URL을 브라우저로 열어 본다

표에 번호 · 제목 · 모집기관 · 상태 · 접수기간 · 등록일이 보이고, 위에 상태 필터(접수예정 · 접수중 · 마감 · 결과공고)와 검색이 있습니다.

2. URL은 다음 장 프롬프트에 붙인다

결과물은 채용/ 폴더에 쌓이므로 두 터미널을 채용/으로 옮깁니다.

3. 폴더에 이미 있는 것

채용.csv는 게시판 표 한 쪽(10행)을 옮긴 데이터. 재생성 때 프로그램이 읽습니다. 키트에 들어 있습니다.

확인 포인트 URL을 열어 표와 상태 필터를 봤고, ls에 채용.csv가 보인다.

[서버 터미널] · 예제 B 폴더 · 레퍼런스 URL

```
$ cd ~/week02/채용
$ ls
채용.csv
```

[레퍼런스 URL · 전원 같은 페이지 · 프롬프트에 붙임]

<https://www.gjcity.go.kr/portal/examInfo/list.do?mId=0202050000>

광주시청 · 채용시험정보 · 표 10행 · 상태 필터 · 검색 · 쪽 이동

[채용.csv 형식 · 키트에 포함]

모집기관, 제목, 상태, 접수기간, 등록일

참고 상세 보기는 스크립트 링크라 AI가 목록 한 쪽만 읽습니다. 명세를 뽑기에는 충분합니다. 안 읽히면 A 3/8의 폴백(curl로 받아 페이지.html)과 같습니다.

URL을 읽고 Plan에서 구조 → 기능 → 명세

A와 같은 두 프롬프트입니다. 근거는 프롬프트에 붙인 광주시청 채용시험정보 페이지. 둘 다 Plan 모드에서.

1. [Claude Code 터미널] 예제 B 폴더에서 claude

Shift+Tab 두 번으로 Plan 모드. 첫 실행이면 폴더 신뢰 Yes.

2. URL을 읽고 구조 추정 → 기능 목록

페이지에서 보이는 것(광고 목록 · 상태 필터 · 검색 · 쪽 이동)이 기능으로 잡힙니다. 목록만 읽으므로 상세 화면의 내용에는 「추측」이 붙어야 합니다. 안 읽히면 A 3/8 풀백.

확인 포인트 화면 목록 · 이동 흐름 · 「추측」 표시가 든 구조 추정과 기능 목록이 화면에 있다.

```
[Claude Code 안] · Plan 모드 · 두 프롬프트 (A와 같음)

$ cd ~/week02/채용
$ claude
# Shift+Tab 두 번 → 하단에 plan 표시
>
https://www.gjcity.go.kr/portal/examInfo/list.do?mId=0202050000
페이지의 채용시험정보 게시판 목록을 대상으로 구조를 추정하라. 완료 조건: ①
목록에 있는 정보 항목과 화면에서 할 수 있는 조작(상태 필터 · 검색 · 쪽 이
동) ② 화면 뒤에서 일어나는 처리의 추정 목록 ③ 목록만으로 알 수 없는 것의
목록. 게시판 밖의 다른 메뉴는 대상이 아니다. 페이지에 없는 내용은 추측이라
고 표시하라.

> 이 서비스의 기능 목록을 만들어라. 완료 조건: 각 기능은 한 줄 설명과 관련
화면 이름을 포함하고, 화면 기능과 내부 처리 기능을 구분한다.
```

결과 비교 같은 URL인데 화면 목록 · 기능 개수는 사람마다 다릅니다. A와 같은 모양(목록 · 조건으로 찾기)이 나오는지 봅니다.

명세 저장 → EARS 라벨 → 읽기

명세는 파일이어야 AI가 읽고 우리가 고칠 수 있고, EARS 라벨이 붙어야 검증할 수 있는 문장이 됩니다. 저장은 파일을 만드는 일이므로 Plan을 해제합니다.

1. Shift+Tab으로 Plan 해제

하단의 plan 표시가 꺼집니다.

2. 두 프롬프트를 이어서 보낸다

명세를 파일로 저장하고 EARS 라벨을 붙입니다.

3. 읽고 이해한다

VS Code 탐색기에서 spec/요구사항명세서.md를 클릭해 열고 REQ를 위에서부터 읽습니다. Ctrl+Shift+V로 md 파일을 가독성있게 볼 수 있습니다.

4. 개선할 부분을 찾아본다

내가 쓸 프로그램이라면 어떻게 할지 생각하며 읽습니다. 고치고 싶은 문장이 있으면 직접 고치거나 AI의 도움을 얻어 고칩니다(선택). 없으면 그대로 둡니다.

확인 포인트 spec/에 라벨이 붙은 명세서, 고칠 곳이 있었으면 ← 고침, 내 소재.md에 질의 ①④의 답이 있다.

[Claude Code 안] · 해제 후 두 프롬프트 (A와 같음)

> 요구사항 명세서를 만들어라.

- 근거: 위 기능 목록
- 형식: 한 문장 (조건 + 할 일) · 고유 ID · 관련 화면
- 저장: spec/요구사항명세서.md
- 다시 만들 프롬프트: spec/프롬프트.md

> 각 문장 앞에 EARS 문형 라벨을 붙여 다시 저장하라.

- 라벨: 상시 · 사건 · 상태 · 원치 않는 동작 · 선택
- 안 맞는 문장은 고쳐서 라벨을 붙여라
- 보고: 고친 문장 목록

명세 읽기 질의 ①④ · 답은 내소재.md에

① 확인할 수 없는 말("잘", "적절히", "필요 시")이 든 REQ 하나와 내가 고친 문장

④ 내 프로그램에 적용하고 싶은 부분 한 곳 (예: 시작할 때 접속중만 보기) → 문장을 바꾸고 ← 고침

결과 예상 [사건] 사용자가 상태를 고르면, 시스템은 그 상태의 공고만 표시해야 한다 / [상시] 시스템은 공고마다 모집기관 · 접수기간을 표시해야 한다.

재생성 → 명세 추가 → 다시 만들기

페이지도 코드도 없이 명세서 한 장으로 프로그램이 나오는지 보는 것이 재생성이고, 프로그램을 코드가 아니라 명세로 바꾸는 것이 발전입니다. A에서 한 번 본 것을 다른 재료로 확인합니다.

1. rebuild에 명세서 · 채용.csv만 → CLI 재생성

앞 장의 Claude Code는 그대로 둔 채, 터미널을 하나 더 열어 rebuild에서 claude를 띄우며(폴더 신뢰 Yes) A와 같은 프롬프트를 보냅니다. 예상: 「1) 공고 목록 2) 상태로 찾기 3) 검색」 정도의 메뉴가 있는 채용.py. 서버 터미널에서 실행해 메뉴를 눌러 봅니다.

2. 명세 끝에 한 줄 ← 추가

실행해 보고 내 조건으로 찾을 수 있는지 봅니다(예: 접수중인 공고만). 안 되면 그것이 명세에 더할 한 줄입니다. 38장의 갈래표를 그대로 씁니다. 관심 있는 공고 한 건은 프롬프트 「」에 넣어 AI가 채용.csv에 추가하게 합니다(VS Code로 열어 확인 · 수정 가능).

3. 다시 구현 → 실행

예상: 추가한 줄이 메뉴나 표시에 나타나고, 내가 넣은 공고가 목록에 보입니다. 코드는 손대지 않았습니다.

확인 포인트 채용/rebuild/의 프로그램이 실행된다. ← 추가와 새 공고를 넣었다면 그것까지 반영된다.

● ● ● [새 Claude Code 터미널] · 재생성 → [서버 터미널] · 실행

```
$ cd ~/week02/채용
$ mkdir -p rebuild
$ cp spec/요구사항명세서.md 채용.csv rebuild/
$ cd rebuild
$ claude
# claude가 뜨면 여기부터 Claude Code 안
> 이 폴더의 요구사항명세서.md만을 근거로, python3 하나로 실행되는 명령줄 프로그램(CLI)을 구현하라. 데이터는 같은 폴더의 채용.csv를 읽고, 채용.csv로 할 수 있는 기능만 만든다. 명세에 없는 내용은 추측하지 말고 부족한 정보는 질문 목록으로 정리해 보고하되, 답을 기다리지 말고 명세에 있는 것만으로 구현하라.
# [서버 터미널]
$ cd ~/week02/채용/rebuild
$ ls
$ python3 (파일이름).py
```

● ● ● 명세서에 덧붙일 예 → [Claude Code 안] · 반영 → 실행

```
[상태] 접수기간이 지나는 동안, 시스템은 그 공고를 "마감"으로 표시해야 한다. ← 추가
$ claude --continue
> 요구사항명세서.md에 ← 추가 표시된 요구를 반영하고, 채용.csv에 「모집기관, 제목, 상태, 접수기간」 공고 한 건을 추가한 뒤 다시 구현하라. 명세에 없는 것은 추측하지 말고 질문하라.
# [서버 터미널]
$ python3 (파일이름).py
```

같은 절차가 회사에서 쓰이는 자리

A · B에서 한 일은 「남의 서비스에서 명세를 뽑아, 필요하면 고쳐, 명세만으로 만든다」였습니다. 회사에서는 이렇게 쓰입니다.

1. 경쟁 서비스 분석 → 우리 기획서

경쟁사 페이지 URL을 읽혀 화면 · 기능 목록을 뽑고, 우리 서비스에 맞게 REQ를 고쳐 기획 초안으로.

2. 외부 페이지를 사내 조회 도구로

늘 들어가 보는 외부 페이지(관세청 고시, 협회 공고, 거래처 가격표)를 명세로 뽑아 우리가 쓰는 조건으로 걸러 보는 도구로. B의 시청 게시판 → 상태로 찾기 CLI

3. 요구 변경을 코드가 아니라 명세로

고객이 "마감된 건은 빼 주세요"라고 하면 명세에 [상태] 한 줄을 추가하고 다시 만듭니다. 변경 이력이 명세에 남습니다.

4. 신입에게 서비스 설명하기

명세서가 온보딩 문서를 겸합니다. (다음 주 기계가 검증하는 TDD로 이어집니다.)

● ● ● 한 줄로 · 오늘 절차와 회사 일의 대응

URL 하나 → 명세 경쟁 분석 · 벤치마킹 문서
EARS 라벨 기획서의 요구사항 표 · QA 체크리스트
읽고 ← 고침 우리 서비스 기획 · 리뷰 코멘트
명세만으로 재생성 외주 · 신규 인력에게 넘기는 발주서
명세에 ← 추가 변경 요청서 · 변경 이력
빈틈 찾기 인수 테스트 · 사용자 검수

다음 예제 C 근처 맛집(교수 데모). URL로 읽히지 않는 서비스는 화면을 보는 AI가 근거를 만듭니다.

URL로 읽히지 않는 서비스는?

URL로 읽히지 않는 서비스, 예를 들어 지도는 사람이 화면을 보고 적어서 근거를 만들었습니다. 오른쪽이 메모 예시입니다. 원래는 맵에서 「경기광주역 맛집」을 검색해 중요한 사항을 확인하며 써야 합니다.

1. 무엇을 적나

화면의 이름과 순서, 자주 하는 동작, 가게 하나에 붙는 정보 항목, 쓰면서 불편했던 것. 명세를 뽑을 때 AI가 물어볼 것들을 미리 답해 두는 것입니다.

2. 왜 어려운가

빠뜨리고, 사람마다 다르게 적고, 시간이 걸립니다. 그리고 적는 사람이 이미 서비스를 해석해 버려서, AI는 화면이 아니라 사람의 해석을 근거로 삼게 됩니다.

3. 지금은

이 메모를 쓰는 일을 화면을 보는 브라우저 AI에게 맡깁니다. 무엇을 적어야 하는지는 그대로이고, 적는 주체만 바뀝니다.

● ● ● [사람이 쓴 관찰 메모 예시] 구글맵 정보

「경기광주역 맛집」 검색

[화면] 검색창 → 결과 목록 → 가게 상세 → 길찾기 / 즐겨찾기

[동작] 검색어 입력 · 결과 카드 누르기 · 지도 이동 · 저장

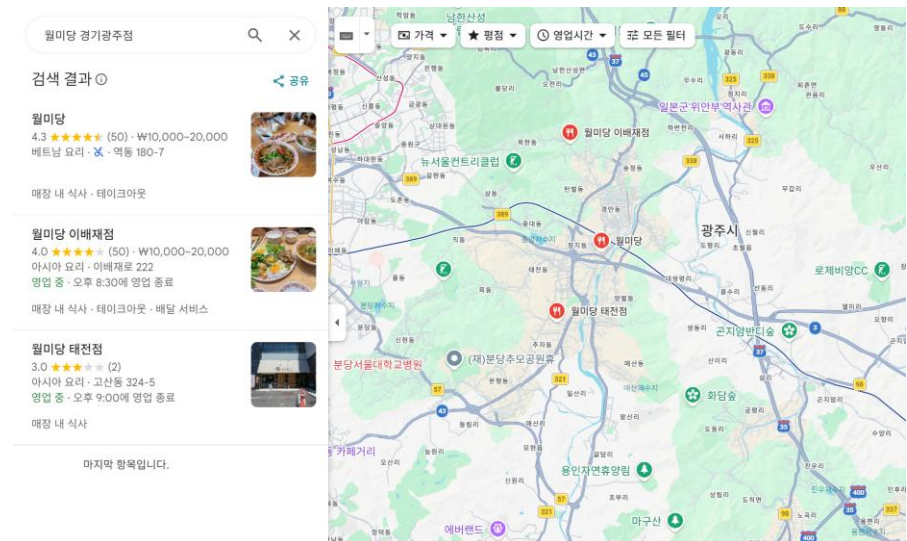
[가게 정보] 이름 · 종류 · 별점 · 리뷰 수 · 거리 · 영업시간 · 주소 · 전화

[정렬 · 필터] 정확도 / 거리 / 인기 · 영업중만 보기

[불편] 걸어서 몇 분인지 바로 안 보인다 · 광고 가게가 위에 섞인다

[알 수 없음] 별점이 어떻게 계산되나 · 광고와 일반 결과의 기준

[내가 아는 가게] 월미당 태전점 · 아시아 요리 · 고산동 · 별점 3점



화면을 직접 눌러 보는 AI가 근거를 만든다

A · B는 AI가 URL을 읽어 근거를 얻었습니다. 지도 앱은 다릅니다. 이 예제는 실습이 아니라 데모입니다 (Mac 전용 AI 브라우저 사용).

1. 왜 URL로는 안 되나

지도 앱은 화면을 JavaScript로 그립니다. URL을 읽으면 빈 껍데기만 옵니다. 검색 결과 · 가게 상세 · 거리는 사람이 눌러야 나타납니다.

2. 예전: 사람이 관찰 메모를 쓴다

앞 장의 메모가 그것입니다. 무엇을 적어야 하는지는 알지만 범위가 지나치게 넓고, 시간이 걸리고, 사람마다 다르고, 주관이 들어가고, 빠뜨립니다.

3. 지금: 화면을 보는 AI가 메모를 쓴다

브라우저 안에서 화면을 조작하며 구조를 읽는 AI를 사용합니다.

[두 가지 근거 만들기] · 같은 항목, 다른 주체			
	A · B	C (예전)	C (지금)
근거 읽는 주체	웹 페이지	지도 앱 화면	지도 앱 화면
방법	WebFetch (AI)	사람	브라우저 AI
결과	URL 한 번 읽기	10분 관찰 · 메모	써 보며 읽기
그 뒤	구조 · 기능	서비스관찰.md	서비스관찰.md
	같은 여섯 단계	같은 여섯 단계	같은 여섯 단계

핵심 근거가 URL이든 메모든 AI 관찰이든, 그 뒤 단계는 같습니다.

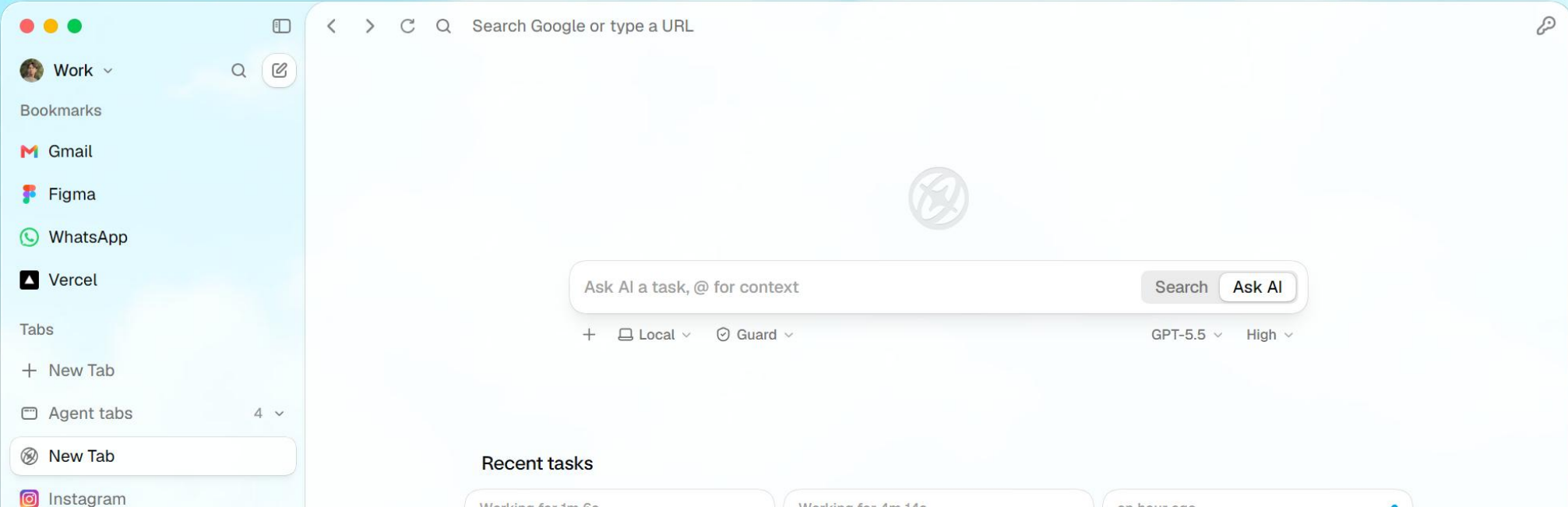


Join waitlist

Backed by Y Combinator >

The most intelligent AI assistant, but it's a browser.

↓ Join waitlist



Aside

다운 브라우저 설치

커머스 분석

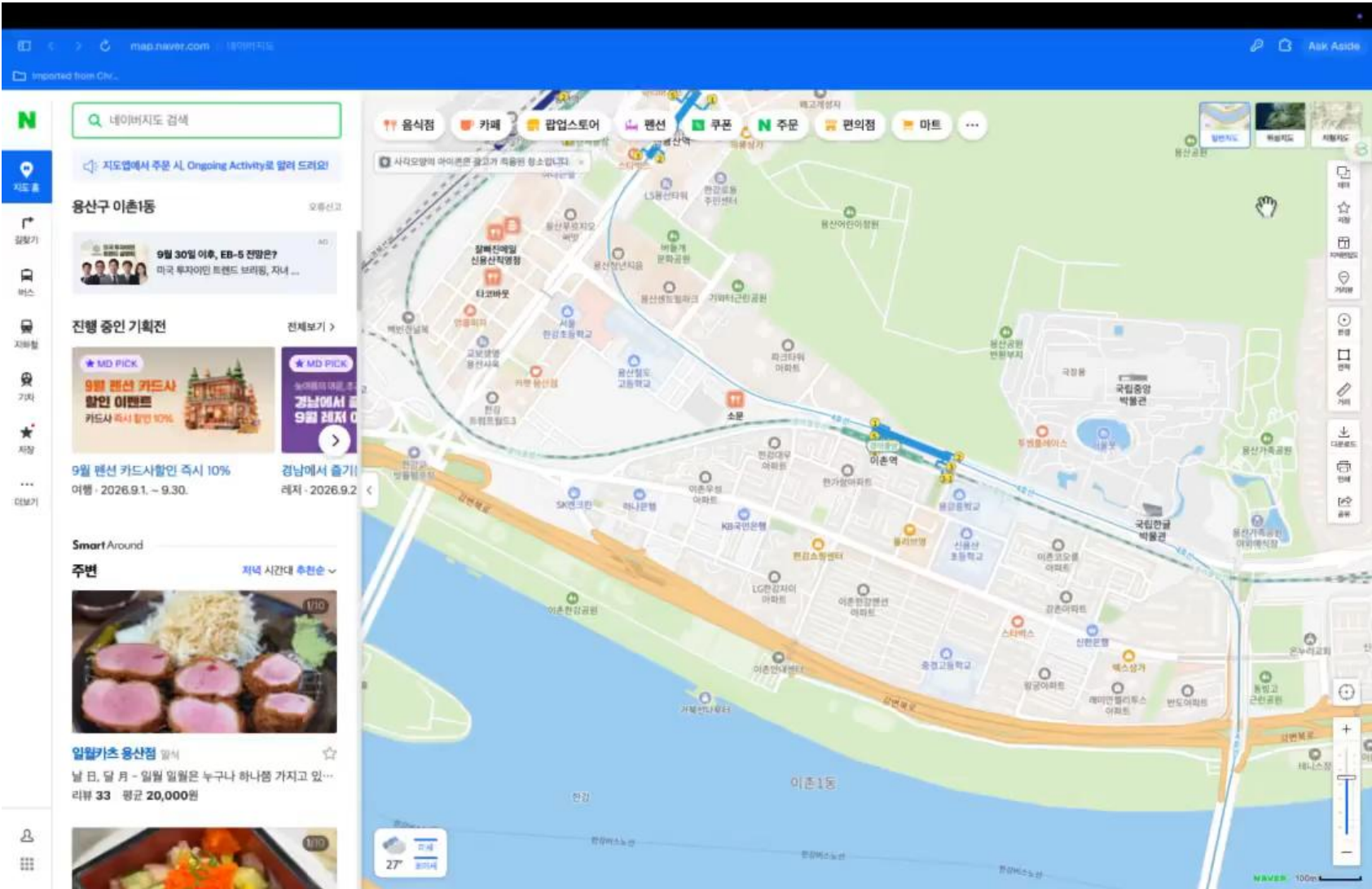
내장악 분석

접차도 뚝뚝

**네이버 블로그까지 뚫는
미친 AI브라우저 10분컷**

화면을 직접 조작하는 AI에게 관찰을 맡긴다

URL을 읽는 AI는 지도 앱 앞에서 멈춥니다. 검색창에 입력하고, 결과 카드를 누르고, 탭을 도는 조작을 사람 대신 합니다.



지도 화면 → 서비스 구조 분석서

A · B · C의 첫 프롬프트는 요청 구조가 같습니다.

세 예제의 공통 구조

- 분석할 범위를 먼저 못 박는다
- 완료 조건: ① 있는 것 ② 추정 ③ 알 수 없는 것
- 명세가 아니라 구조 분석서를 요청한다
- 추측은 추측이라고 표시하게 한다

C만 다른 이유: 지도는 동적 화면

A · B는 한 화면에 다 보이는 표와 목록이라 「무엇이 적혀 있나」를 물으면 됩니다. 지도는 눌러야 다음 화면이 나오므로 「화면이 몇 개이고 어떻게 이어지나」를 물어야 합니다.

- ①이 화면 목록과 이동 흐름 (A · B는 정보 항목)
- 누른 만큼이 근거다. 안 누른 곳은 명세에 들어오지 않는다

질문 내 소재는 URL로 읽히는가, 눌러야 바뀌는가?

누려야 바뀌면 조작하는 AI로 근거를 만듭니다.

확인 키트의 맛집/폴더에 구조분석 문서가 있습니다.

● ● ● [브라우저 AI에 입력] · 첫 프롬프트

> 지금 열린 지도 앱에서 「경기광주역 맛집」을 검색하고 결과 3곳을 직접 눌러 본 뒤, 이 서비스의 구조를 추정하라. 완료 조건: ① 화면(섹션) 목록과 이동 흐름 ② 화면 뒤에서 일어나는 처리의 추정 목록 ③ 써 보고도 알 수 없는 것의 목록. 직접 보지 못한 것은 추측이라고 표시하라.

● ● ● [산출물] · 네이버지도-구조분석.md

0. 수행한 조작 7단계
1. 화면 8개와 이동 흐름
2. 화면 뒤 처리 추정
3. 알 수 없는 것
4. 정보 항목 9묶음
5. 실측값
6. 한 줄 요약

구조 → 기능 → 명세 → 재생성

구조 분석서가 나온 뒤는 절차가 동일합니다.

1. 기능 목록

각 기능에 한 줄 설명과 관련 화면 이름을 달고, 화면 기능과 내부 처리 기능을 구분합니다.

2. 요구사항 명세서

조건과 행위가 드러나는 한 문장, 고유 ID, 관련 화면. spec/ 폴더에 명세서와 프롬프트를 함께 저장합니다.

3. EARS 문형 분류

다섯 문형 라벨을 붙이고, 어느 문형에도 들어가지 않는 문장은 고쳐 다시 저장합니다.

4. 명세만으로 재생성

명세에 없는 것은 만들지 않게 합니다. 못 채운 칸은 명세로 되돌려 한 바퀴를 마칩니다.

질문 내 명세만 보고 다시 만들면 어디가 비는가?
빈 칸이 다음에 채울 근거입니다.

[Claude 프롬프트]

- ② 기능 목록을 만들어라
 - 한 줄 설명 + 관련 화면, 화면/내부 처리 구분
- ③ 요구사항 명세서를 작성하라
 - 조건·행위 한 문장, 고유 ID, 관련 화면
 - spec/요구사항명세서.md · spec/프롬프트.md
- ④ EARS 다섯 문형으로 분류하고 고쳐 저장하라
- ⑤ 명세만 보고 다시 만들어라
 - 명세에 없는 것은 만들지 마라 · 맛집.csv만 → rebuild/

[터미널에서 실행]

```
[baki@aisw02 09/09 19:07 ~]$ cd ~/week02/맛집
[baki@aisw02 09/09 21:07 맛집]$ ls
캡처          네이버지도-구조분석.md  rebuild
맛집.csv      서비스관찰.md           spec
[baki@aisw02 09/09 21:07 맛집]$ cd rebuild
[baki@aisw02 09/09 21:07 rebuild]$ python3 맛집.py
```

명세만으로 다시 만든 결과

원본은 지도인데 결과는 **메뉴가 있는 프로그램**입니다. A와 달리 **고를 수 있는 것은 명세에 「사건」 문형**이 있었기 때문입니다.

1. 기능은 세 가지

가게 목록 · 도보 시간 순 · 종류로 찾기. 없는 번호와 없는 종류도 안내로 받아 냅니다. 명세에 적힌 것만 있습니다.

2. 도보 시간 순은 원본에 없다

지도에서는 거리가 가게에 붙어 있지 않아 길찾기를 열어야 알 수 있었습니다. 그 불편을 REQ-09로 세웠습니다.

3. 지도 · 리뷰 · 영업중은 없다

범위 밖으로 선언한 대로입니다. 없는 것을 지어내지 않은 결과가 이 화면입니다.

확인 키트 맛집/rebuild/ 에 결과물이 들어 있습니다.
실행하지 않고 파일만 열어도 됩니다.

서버 터미널 — 예제 C 재생성 결과

```
$ cd ~/week02/맛집/rebuild
$ ls
  맛집.csv   맛집.py   요구사항명세서.md
$ python3 맛집.py
```

1) 가게 목록 2) 도보 시간 순 3) 종류로 찾기 0) 종료
선택: 1

[가게 목록]

- 1) 정원갈비 경기광주점 | 갈비 | 경기광주역에서 도보 12분 | 15000원 이상 · 별점 4.85
 - 2) 후라토식당 경기광주점 | 일식 | 경기광주역에서 도보 15분 | 10000~15000원
 - 3) 고반식당 광주경안점 | 고기요리 | 경기광주역에서 도보 8분 | 15000원 이상 · 예약 가능
 - 4) 학교 앞 김밥집 | 분식 | 정문에서 도보 3분 | 5000원 이하 · 점심시간에 줄이 김
 - 5) 경기광주역 국밥 | 한식 | 경기광주역에서 도보 5분 | 8000~10000원 · 혼밥 가능
- 모두 5곳

선택: 2

[도보 시간 순]

- | | |
|---------------------|---------------------|
| 1) 학교 앞 김밥집 · 도보 3분 | 2) 경기광주역 국밥 · 도보 5분 |
| 3) 고반식당 · 도보 8분 | 4) 정원갈비 · 도보 12분 |
| 5) 후라토식당 · 도보 15분 | |

선택: 9

없는 번호입니다. 1 · 2 · 3 · 0 중에서 고르세요.

선택: 3 종류 (예: 분식, 한식, 카페): 중식

'중식' 종류의 가게가 없습니다. 있는 종류: 갈비, 고기요리, 분식, 일식, 한식

예제 C · 보충 · 왜 다르게 했나

이 화면에서 가져올 수 있는 것

캡처가 알려 주는 것은 그 순간 무엇이 보였는지까지입니다. 다시 만들려면 화면을 채울 값과 규칙이 필요합니다.

1. 지도는 그려진 이미지로 온다

핀과 도로가 보이지만 좌표 · 도로 · 건물 데이터는 화면에 없습니다. 캡처는 그림이지 데이터가 아닙니다.

2. 리뷰는 건수와 한 조각만 보인다

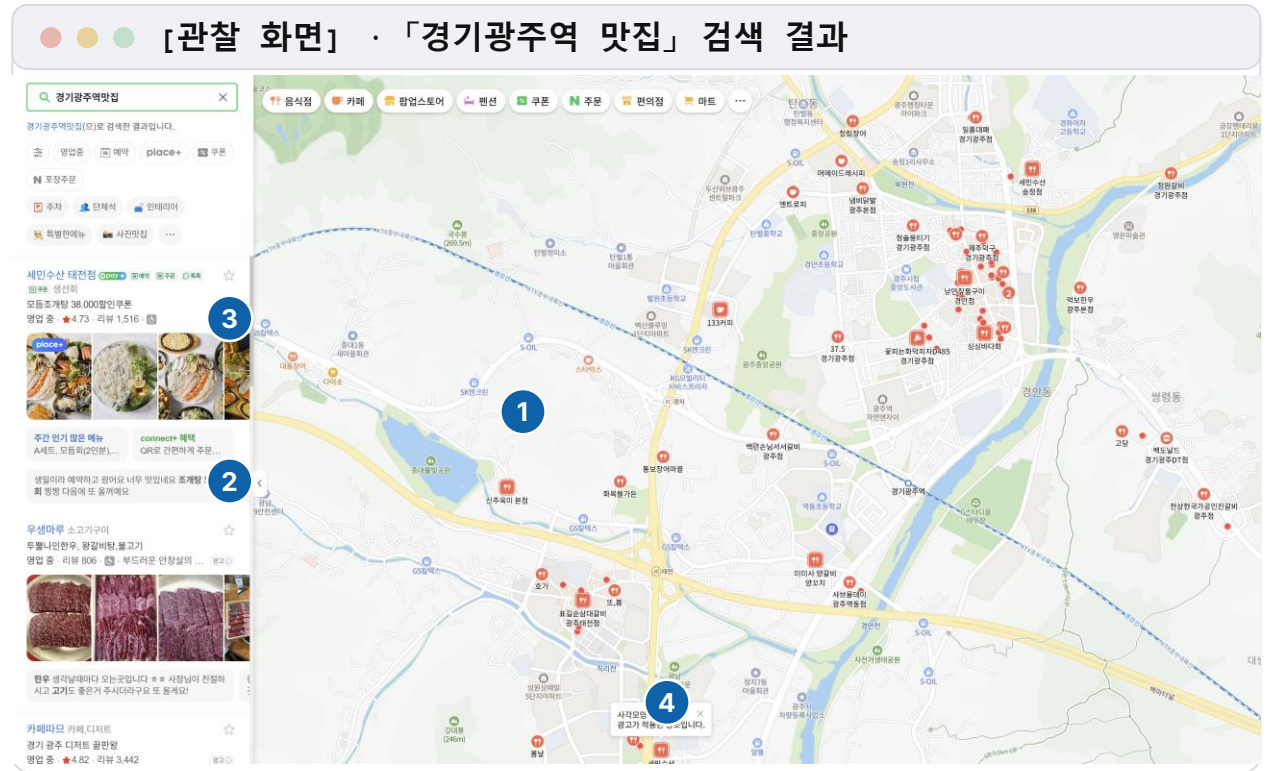
목록에는 「리뷰 1,516」과 한 줄뿐입니다. 상세에서도 3,480건 중 12건만 노출됐습니다.

3. 「영업 중」은 저장된 값이 아니다

영업시간표와 지금 시각을 비교해 매번 만드는 표시입니다. 19:30의 값을 적어 두면 새벽 3시에도 영업 중이 됩니다.

4. 순위는 규칙을 알 수 없다

「광고」 배지가 상단에 섞이고, 다시 불러오면 순서가 바뀝니다. 무엇을 얼마나 반영하는지 화면에 설명이 없습니다.



판정 값을 끝까지 채울 수 있나 · 내일도 참인가 · 내가 만들어도 되나
하나라도 아니오면 범위 밖. 맛집.csv 여섯 열은 셋 다 예입니다.

명세가 못 만들 것을 먼저 걸러 냈다

지도에서 뽑은 **결과가 메뉴 프로그램**인 것은 명세가 부실했기 때문이 아닙니다. 없는 것은 가게 좌표 · 실시간 영업정보 · 리뷰였고, 손으로 코딩해도 얻어지지 않습니다.

1. 만들기 전에 빈 곳을 찾아냈다

EARS 라벨 보고의 「빈 문형: 상태」. csv에 영업시간이 없어 [상태]만 비었습니다. 문형의 빈칸이 근거의 빈칸을 가리킵니다.

2. 못 만드는 것을 문서에 남겼다

개요 첫 문단이 지도 · 사진 · 리뷰 · 예약 · 로그인을 범위 밖으로 잘라냈습니다. 「왜 지도가 없냐」에 답할 문서가 그것입니다.

3. 다음에 할 일을 가리켰다

영업시간 열을 더하면 [상태] 문장이 살아납니다. 근거를 늘리면 범위가 넓어집니다.

● ● ● [빈 문형이 다음 할 일을 가리킨다]

[상태] 문형이 비어 있다

↑ 왜 맛집.csv에 영업시간 열이 없다

→ 하면 열 하나를 더하면 문장이 살아난다

[상태] 영업 중인 동안, 시스템은
「영업중」을 표시해야 한다

● ● ● [명세가 없었다면]

명세 없이: 「지도 앱 같은 걸 만들어 줘」

→ 가짜 지도와 지어낸 가게 정보. 쓸 수는 없다

명세로: 범위 밖을 먼저 적는다

→ 만든 것에는 근거가 있다

요점 못 만드는 줄 모르고 만들어 두는 것이 실패입니다.
문장이 못 담는 「쓰는 느낌」은 화면 명세와 TDD로 이어집니다.

준비: 포트폴리오 저장소를 original로 받는다

근거는 잘 만든 개인 포트폴리오 깃허브 저장소입니다. 결과물이 화면으로 나오는 첫 예제이고, 이름과 프로젝트만 바꾸면 여러분 포트폴리오로 쓸 수 있습니다.

1. 왜 이 저장소인가

루트에 index.html 한 장과 assets 폴더뿐. 서버에서 clone만 하면 바로 열립니다. 소개 · 프로젝트 · 연락처가 한 화면에 있어 취업용으로 좋습니다.

2. 예제 D 폴더에서 clone

두 터미널 각각 `cd ~/week02/포트폴리오`. `ls original`에 index.html이 보이면 됩니다.

3. 2가 안 되면 키트 파일로

키트의 index.html을 original/에 복사합니다.

[서버 터미널] · 저장소 받기

```
$ cd ~/week02/포트폴리오
$ git clone https://github.com/bedimcode/responsive-portfolio-website-patrick original
$ ls original
README.md  assets  index.html  preview.png
```

[서버 터미널] · git clone이 안 될 때만

```
$ mkdir -p original && cp index.html original/
$ ls original
index.html
```

확인 포인트 ~/week02/포트폴리오/original 에 index.html이 있다.

결과 검토 ls original에 index.html이 보이면 준비 완료.

Plan에서 코드 구조 분석 → 기능 목록

근거만 "original 폴더의 코드"로 바꿉니다. 페이지가 아니라 소스파일 자체를 읽으므로 결과가 훨씬 구체적입니다.

1. [Claude Code 터미널] 예제 D 폴더에서 claude

original이 보이는 위치에서 실행해야 Claude가 읽을 수 있습니다. Shift+Tab 두 번으로 Plan 모드. 첫 실행이면 폴더 신뢰 Yes.

2. 구조 분석 → 기능 목록

완료 조건을 붙인 두 프롬프트. 근거가 코드라서 기능마다 관련 파일 경로를 지목할 수 있습니다. 화면만 본 A~C는 기능이 어디서 나오는지 「추측」으로 남겼지만, 이번에는 확인이 됩니다. 다음 장의 명세와 4/7 재생성에서 볼 곳도 이 경로가 가리킵니다. 파일은 수정하지 않습니다(Plan).

확인 포인트 파일 경로가 붙은 기능 목록과 화면/내부 구분이 화면에 있다.

[Claude Code 안] · Plan 모드 · 두 프롬프트 (근거만 코드)

```
$ cd ~/week02/포트폴리오
```

```
$ claude
```

```
# Shift+Tab 두 번 → 하단에 plan 표시
```

```
> original 폴더의 프로젝트 전체 구조를 분석하라. 완료 조건: ① 폴더·주요 파일의 역할 목록 ② 사용 기술 스택 ③ 프로그램의 진입점과 실행 흐름 요약. 파일은 수정하지 마라.
```

```
> 이 프로젝트의 기능 목록을 만들어라. 완료 조건: 각 기능은 한 줄 설명과 관련 파일 경로를 포함하고, 화면 기능과 내부 처리 기능을 구분한다.
```

- 구조: index.html 한 파일 · assets/css · assets/js · 스택: HTML·CSS·JS

- [화면 기능] F1 메뉴로 섹션 이동 · F2 프로젝트 카드 · [내부] F4 스크롤 위치에 따른 메뉴 강조

결과 비교 코드를 읽은 명세는 URL을 읽은 명세보다 「추측」이 적습니다. (대신 화면이 어떻게 보이는지는 코드만으로도 다 적히지 않습니다.)

명세 저장 → EARS 라벨 → 읽기

명세는 파일이어야 AI가 읽고 우리가 고칠 수 있고, EARS 라벨이 붙어야 검증할 수 있는 문장이 됩니다. 저장은 파일을 만드는 일이므로 Plan을 해제합니다.

1. Shift+Tab으로 Plan 해제

하단의 plan 표시가 꺼집니다.

2. 두 프롬프트를 이어서 보낸다

명세를 파일로 저장하고 EARS 라벨을 붙입니다.

3. 읽고 이해한다

VS Code 탐색기에서 `spec/요구사항명세서.md`를 클릭해 열고 REQ를 위에서부터 읽습니다. **Ctrl+Shift+V**로 md 파일을 가독성있게 볼 수 있습니다.

4. 개선할 부분을 찾아본다

내가 회사에 낼 포트폴리오라면 어떻게 할지 생각하며 읽습니다. 고치고 싶은 문장이 있으면 직접 고치거나 AI의 도움을 얻어 고칩니다(선택). 없으면 그대로 둡니다.

확인 포인트 포트폴리오/spec/에 라벨이 붙은 명세서, 고칠 곳이 있었으면
← 고침, 내소재.md에 질의 ①④의 답이 있다.

[Claude Code 안] · 해제 후 두 프롬프트 (A와 같음 · 저장 위치만 명시)

> 요구사항 명세서를 만들어라.

- 근거: 위 기능 목록
- 형식: 한 문장(조건 + 할 일) · 고유 ID · 관련 파일
- 저장: claude를 띄운 폴더의 `spec/요구사항명세서.md` (original/안이 아니다)
- 다시 만들 프롬프트: `spec/프롬프트.md`

> 각 문장 앞에 EARS 문형 라벨을 붙여 다시 저장하라.

- 라벨: 상시 · 사건 · 상태 · 원치 않는 동작 · 선택
- 안 맞는 문장은 고쳐서 라벨을 붙여라
- 보고: 고친 문장 목록

명세 읽기 질의 ①④ · 답은 내소재.md에

① 확인할 수 없는 말("잘", "적절히", "필요 시")이 든 REQ 하나와 내가 고친 문장

④ AI가 스스로 정한 항목 중 내가 동의하지 않는 한 곳 (고치고 ← 고침 표시)

결과 비교 라벨이 붙은 REQ의 개수와 「원치 않는 동작」 문형의 유무를 확인합니다.

명세만으로 다시 만들기

1. rebuild 폴더에 명세서만 복사합니다

원본 코드는 넣지 않습니다. 앞 장의 Claude Code는 그대로 둔 채, 터미널을 하나 더 열어 예제 D 폴더에서 시작합니다.

2. rebuild에서 claude 실행, 재생성 요청

새 폴더의 첫 실행이라 폴더 신뢰를 물으면 Yes, 하단에 plan 표시가 없는지 확인합니다. **HTML 정적 페이지 한 장으로**. 명세에 없는 내용은 추측 금지, 부족한 정보는 질문 목록으로 받습니다.

3. 질문 목록을 봅니다

Claude가 물어 온 것들이 이미 명세의 첫 빈틈 후보입니다. 지금 다 채우지 않아도 됩니다. 나중에 지원할 곳이 정해지면, 여기서 배운 절차로 제출할 포트폴리오를 직접 만들며 그 빈칸을 자기 정보로 채우면 됩니다.

[참고] HTML · CSS · JS

HTML은 내용과 구조, CSS는 배치와 색, JS는 눌렀을 때의 동작입니다. 명세 문장으로 정해지는 것은 주로 HTML의 내용과 JS의 동작이고, 배치와 색은 말로는 정확히 맞춰지지 않아 뒤의 화면 명세(.pen)로 넘어갑니다.

확인 포인트 rebuild/index.html과 "부족한 정보 질문 목록"을 확인했다.

[새 Claude Code 터미널] · rebuild 준비 (셀)

```
$ cd ~/week02/포트폴리오
$ mkdir -p rebuild
$ cp spec/요구사항명세서.md rebuild/
# spec이 original/ 안에 생겼으면 cp original/spec/요구사항명세서.md rebuild/
$ cd rebuild
$ claude
```

[Claude Code 안] · 재생성 요청 (전문 입력)과 응답 예시

> 이 폴더의 요구사항명세서.md만을 근거로 **HTML · CSS · JS 정적 페이지 한 장(index.html)**으로 골격을 구현하라. 명세에 없는 내용은 추측하지 말고 부족한 정보는 질문 목록으로 정리해 보고하되, 답을 기다리지 말고 명세에 있는 것만으로 구현하라.

- 질문: 다크 모드의 처음 상태가 명세에 없습니다 / 프로젝트 카드는 몇 개입니까?
- rebuild/index.html 을 만들었습니다.

결과 비교 원본 코드를 넣지 않았으니 배치·색은 다를 수 있습니다.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!--===== FAVICON =====>
  <link rel="shortcut icon" href="assets/img/favicon.png" type="image/x-icon">

  <!--===== REMIXICONS =====>
  <link href="https://cdn.jsdelivr.net/npm/remixicon@3.0.0/fonts/remixicon.css"
rel="stylesheet">

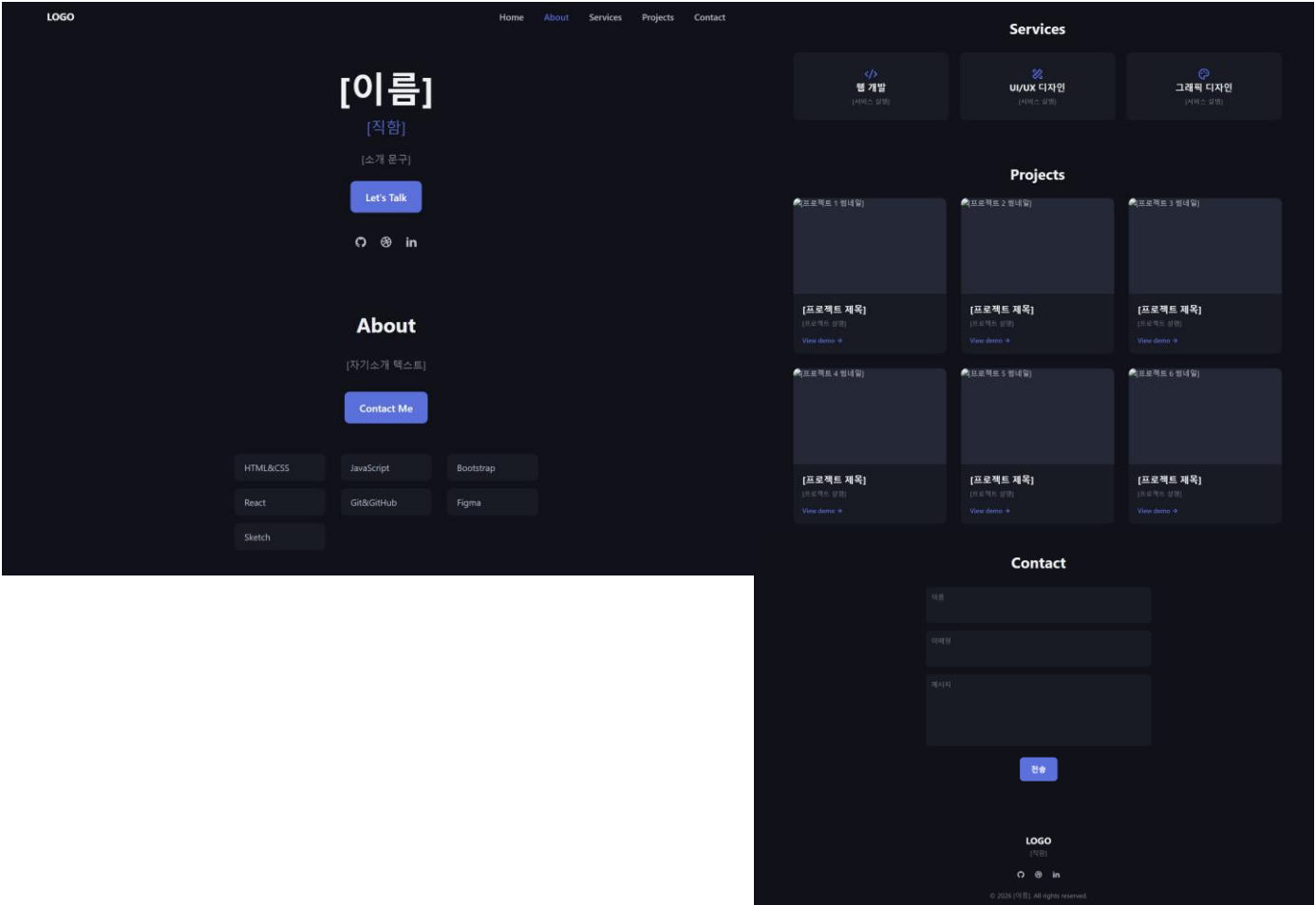
  <!--===== CSS =====>
  <link rel="stylesheet" href="assets/css/styles.css">

  <title>Responsive portfolio website - Bedimcode</title>
</head>
<body>
  <!--===== HEADER =====>
  <header class="header" id="header">

</header>

  <!--===== MAIN =====>
  <main class="main">
    <!--===== HOME =====>
    <section class="home section" id="home">

</section>
```



세 종류의 빈틈: 문장 · 그림 · 사람이 준비하는 것

index.html은 브라우저로 열면 됩니다. 원본과 재생성본을 각각 열어 무엇이 있고 무엇이 비었는지 봅니다.

1. 두 화면을 열어 봅니다

- ✓ 원본은 태그와 주석만 남은 빈 스�কেleton이라 배경색만 보입니다.
- ✓ 재생성본은 메뉴 토글 · 스크롤 · 폼 메시지가 실제로 동작합니다.

2. 차이를 분류합니다

보이는 차이가 기능인지, 배치인지, 자산인지 오른쪽 기준으로 가려냅니다. 고치고 싶은 것이 생기면 7/7에서 한 줄로 씁니다.

가려내는 기준 · 차이가 보였다면 어느 쪽인가

- [기능] 눌렀을 때의 동작 · 보여 주는 정보 · 문구
→ 명세에 문장으로 넣을 수 있다. REQ 한 줄로 채운다
- [화면] 배치 · 간격 · 색 · 순서
→ 문장으로 써도 해석이 같린다. 그림이 정확 → 화면 명세
- [자산] 실제 카피 · 이미지 · 색 값 · 키 → 사람이 준비

기능 · 문구 · 동작의 빈틈은 REQ로 채웁니다.

배치와 색은 화면 명세(.pen)의 대상, 실제 카피와 이미지는 사람이 준비합니다.

무엇이 있고 무엇이 없나 · 세 갈래로

- ✓ 기능 · 동작 original 없음 · rebuild 있음
명세에 문장으로 적혀 있어 그대로 구현
- ✓ 화면 · 배치 original 없음 · rebuild 추정
수치가 명세에 없어 임의로 정함 → .pen으로 고정
- ✓ 내용 · 자산 original 이미지 · 색 변수만 · rebuild 대괄호 자리
글과 사진은 그림으로도 안 정해짐 → 사람이 넣는다

확인 포인트 원본이 왜 빈 화면인지 말할 수 있고, 차이 하나를 기능 · 배치 · 자산 중 어디인지 가려낼 수 있다.

직접 열기 original/index.html과 rebuild/index.html을 각각 브라우저로 엽니다.

발전: 회사에 낼 내 포트폴리오로 만든다

재생성본에는 이름 · 직함 · 소개 · 프로젝트 설명이 대괄호 자리로 남아 있습니다. **회사에 낼 포트폴리오라고 보고 그 자리를 채웁니다.**

1. 낼 내용을 정한다

이름 · 지원 직무 · 소개 한 줄 · 프로젝트 2~3개(한 줄 설명과 링크) · 연락처. 내소재.md 「예제 D」 칸에 적습니다.

2. 아쉬운 기능은 EARS 한 줄로

rebuild/요구사항명세서.md 끝에 조건 + 시스템은 ~해야 한다로 쓰고 ← 추가. 선택입니다.

3. rebuild에서 재생성한다

cd ~/week02/포트폴리오/rebuild → claude --continue. 오른쪽 프롬프트의 「」만 내 정보로 바꿔 보냅니다. 보고에서 ← 추가한 REQ와 내 정보가 반영됐는지 보고, 빠졌으면 그 한 줄만 다시 요구합니다.

4. 이후 기준은 rebuild의 명세서

D의 최신 명세서는 rebuild/요구사항명세서.md, spec/은 추출 당시의 기록입니다. 더 있어야 할 화면 · 기능은 내소재.md 「내 것으로」 칸에.

확인 포인트 rebuild/index.html에 내 이름 · 프로젝트가 반영되어 있고, 내 소재.md에 낼 내용이 적혀 있다.

● ● ● [Claude Code 안] · 프롬프트 하나 (「」 안만 바꿈)

```
> rebuild/요구사항명세서.md에서 ← 추가 표시가 있는 요구사항을 반영하고,  
    소개 · 프로젝트 · 연락처를 「이름 김하늘, 지원 직무 백엔드 신입, 프로젝트 숫자 게임,  
    연락처 github.com/내아이디」로 바꿔 index.html을 다시 구현하라.  
나머지 REQ는 유지하라.  
변경 파일과 반영한 REQ, 사람이 확인할 항목을 보고하라.
```

● ● ● 내소재.md 「예제 D」 · 낼 내용 (예시) · 명세서에 더할 한 줄

```
· 이름      김하늘      · 지원 직무 백엔드 신입  
· 소개      요구사항을 문장으로 정리해 만드는 것을 배웠습니다  
· 프로젝트  숫자 게임 (요구사항 → 테스트 → 구현)  
· 연락처    github.com/내아이디  
[사건] 사용자가 메뉴를 누르면, 시스템은 그 섹션으로 부드럽게 이동해야 한다  
· ← 추가  
# 조건 + 시스템은 ~해야 한다.  
# "잘" · "적절히"가 들어가면 EARS가 아닙니다
```

근거가 코드일 때: 동작은 남고 내용과 배치는 남지 않는다

네 번째도 같은 여섯 단계였습니다. 새로 배운 명령은 git clone 하나. 다른 점은 근거와 결과물입니다.

근거가 코드라서

「추측」이 거의 없었습니다. 코드는 동작을 다 적고 있어 명세가 A~C보다 길고 구체적으로 나옵니다.

결과물이 HTML이라서

원본은 태그와 주석만 있어 화면에 아무것도 안 나오고, 재생성본은 명세만으로 동작하는 한 장이 나왔습니다. 원본은 이미지와 색 변수만 갖고 조립이 없고, 재생성본은 조립을 갖고 글과 사진이 없습니다. 기능은 REQ 한 줄로 채웠지만 배치 · 색과 자산은 그렇게 채워지지 않습니다.

명세가 담는 것과 못 담는 것

명세는 동작과 구조를 담습니다. 배치 · 색은 문장으로 적으면 사람마다 다르게 읽혀 화면 명세(.pen)로 정하고, 글 · 사진 · 키는 문장으로도 그림으로도 정해지지 않아 사람이 넣습니다.

내 것으로

이름 · 프로젝트 · 연락처 세 줄과 ← 추가 한 줄. 이 rebuild/가 다음 절과 3주차의 입력입니다.

● ● ● 되짚어 볼 것 하나 · 답은 내소재.md 「생각 거리」 칸에

원본과 재생성본이 다른 곳 중,
REQ 문장으로 채우기 어려운 것 하나는?

폴더에 남은 것 original/(clone) · spec/(라벨 · ← 고침) · rebuild/(index.html · ← 추가 · 내 정보) · 내소재.md 「내 정보 세 줄」

다음 화면 명세. 말로 주고받으면 갈리는 배치를 그림으로 정하고, 두 명세를 함께 주어 다시 만듭니다.

왜 그림까지 그리나: 말로는 배치가 맞춰지지 않는다

배치는 화면 레이아웃입니다. 섹션과 카드를 어디에 어떤 크기로 놓는가. 기능이 같아도 배치가 다르면 사용자에게는 다른 화면입니다. Pencil은 그 배치를 한 번에 맞추고, 결정을 파일로 남겨 사람과 AI가 같은 것을 보게 하는 도구입니다.

pen.dev · 코드 옆에 그리는 화면 시안

1. 말로 하면 서로 다르게 알아듣는다

「카드를 나란히」가 3열인지 2열인지, 간격이 얼마인지는 사람마다 다릅니다. 예제 D의 재생성본도 명세에 없는 배치와 색을 AI가 임의로 정했고, 다시 만들면 또 달라집니다. 그림 한 장이면 한 번에 맞습니다.

2. 그린 것이 파일로 남는다

캔버스에 그리면 .pen 파일로 저장됩니다. 요구사항명세서.md 옆에 두고 git으로 함께 관리합니다.

3. Claude Code가 그 파일을 읽는다

시안이 말이 아니라 입력이 됩니다. .md와 .pen을 함께 주면 루프가 배치를 추측하지 않습니다. (이 연결을 MCP라고 합니다.)

우리 수업에서

Claude Code가 한 화면을 .pen에 그리고 → 캔버스에서 사람이 고치고
→ rebuild/index.html에 적용
→ 배치와 기능을 눈으로 확인합니다.

색 · 글자 · 여백, 모바일 배치, 다른 섹션 적용은 심화.
디자이너와 같은 파일을 보며 일하는 방식은 참고로 소개합니다.

SDD: 기능은 .md, 배치는 .pen이 기준이 된다

루프의 정보 수집이 두 파일을 읽고, 검증도 두 파일을 기준으로 삼습니다. 사람은 루프 밖에서 두 파일을 쓰고 결과를 실행합니다.

1 기능의 기준 · .md

포트폴리오/rebuild/요구사항명세서.md에서 해당 화면의 REQ를 고릅니다.
예제 D 발전에서 추가한 요구와 내 정보도 유지합니다.

2 배치의 기준 · .pen

Claude Code가 내화면.pen에 한 화면의 섹션·카드·버튼을 배치하고, 캔버스에서 사람이 고칩니다.
문장만으로 해석이 갈렸던 배치를 여기서 정합니다.

3 함께 적용 · 루프에 넣기

Claude Code에 명세와 시안을 함께 줍니다.
서로 충돌하면 임의로 구현하지 않고 먼저 질문하게 합니다. "애매한 명세는 사고"를 막는 장치입니다.

4 직접 검증 · 사람이 루프 밖에서

캔버스와 실제 배치를 대조합니다. 기존 기능과 추가 요구를 실행해 통과·실패·미확인을 기록합니다.

문장 명세 확인 → 화면 설계 → 코드 적용 → 배치와 기능 검증.

말로 설명하기 어려운 것을 그림으로 정하고, AI의 완료 보고만으로 끝내지 않습니다.

Pencil 설치: pen.dev 확장을 서버에 추가

화면 명세 도구 Pencil(pen.dev)을 씁니다. Remote-SSH 창에서 설치해야 확장이 서버 쪽에 들어가고, 서버의 Claude Code가 그것을 봅니다.

1. 확장 탭에서 pen.dev 를 검색한다

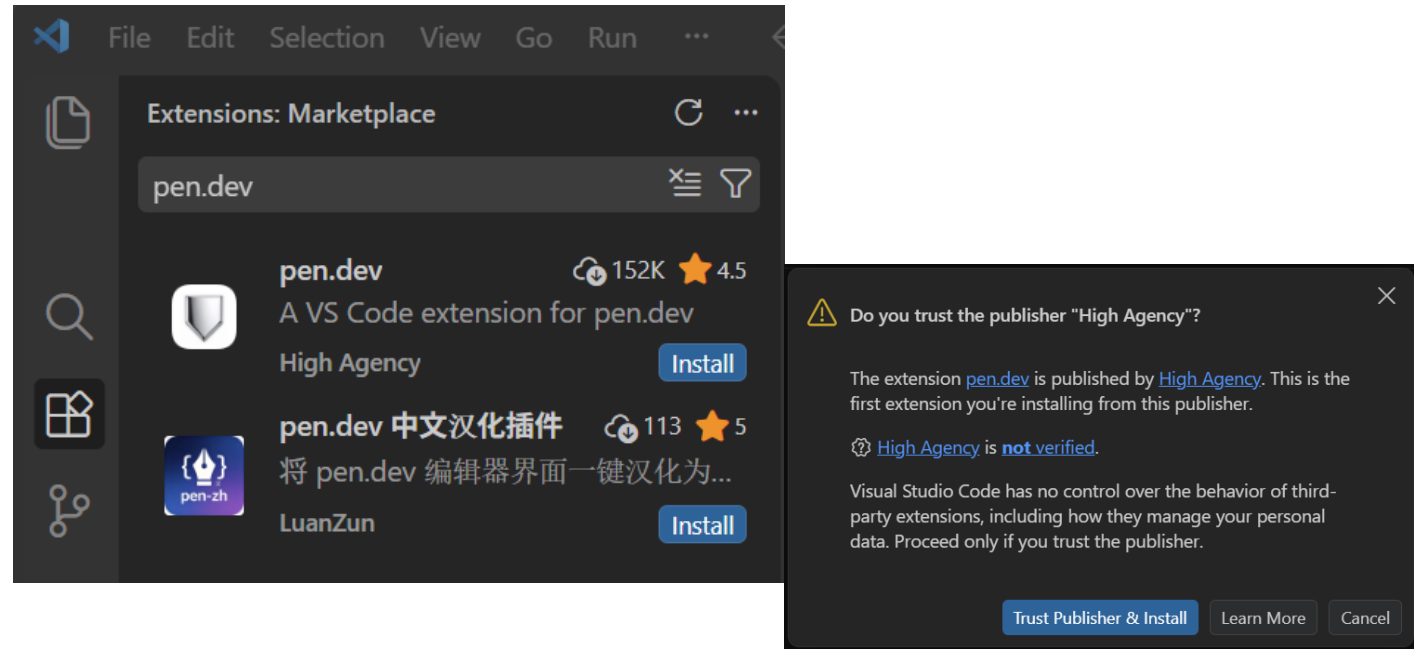
Ctrl+Shift+X → 검색창에 pen.dev. 게시자 High Agency의 것을 고릅니다(아래 중국어 플러그인 아님).

2. Install → "Trust Publisher & Install"

게시자 미인증 경고는 정상입니다. 버튼이 "Install in SSH: aisw02"로 보여야 서버 쪽 설치입니다. "Install"만 보이면 내 PC 창입니다.

3. 설치가 끝나면 왼쪽 막대에 Pen.Dev 아이콘

안 보이면 F1 → Developer: Reload Window. 설치는 금방 끝납니다.



확인 포인트 확장 탭의 pen.dev가 Installed 상태이고, 왼쪽 막대에 Pen.Dev 아이콘이 있다.

참고 이 확장은 내 PC가 아니라 서버(~/.vscode-server)에 설치됩니다. 내 PC 창에서 설치하면 서버의 Claude Code는 보지 못합니다.

pen.dev 로그인: 내화면.pen을 만들고 6자리 코드로

Pencil은 무료지만 계정이 필요합니다. 지금 열 수 있는 메일 주소(gmail 추천)로 로그인합니다.

1. 왼쪽 Pen.Dev 패널 → New .pen file

파일 이름은 **내화면.pen**, 위치는 week02/포트폴리오/rebuild/. 열리면 로그인 화면이 뜹니다.

원격 창(왼쪽 아래 SSH: aisw02)에서 rebuild 폴더를 우클릭해 **내화면.pen** 파일을 만듭니다.

2. 이메일 주소 입력 → Send Code

같은 주소로 로그인 코드 메일이 옵니다. 스팸함도 봅니다.

3. 메일의 6자리 코드를 입력한다

코드는 몇 분 안에 만료됩니다. 만료되면 Send Code를 다시 누릅니다.



Sign in to pen.dev

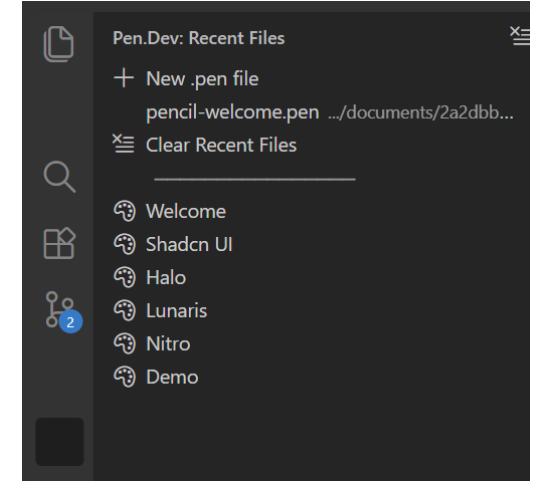
Email Address

내 이메일 주소

Send Code

I have a password

By using this product you agree to our
Privacy Policy, Terms of Use and EULA.



Hi,

You can now log in to [pen.dev](#) using the code below.

6자리 코드 (메일에서 복사)

확인 포인트 Sign in 화면이 사라지고 다음 장의 "Connect to pen.dev via MCP externally" 창이 떴다.

참고 비밀번호는 만들지 않아도 됩니다(코드 로그인). New .pen file이 내 PC 경로를 보여주면 로컬 창입니다.

Claude Code CLI 스위치를 켜 채 Finish

로그인 직후 "Connect to pen.dev via MCP externally" 창이 뜹니다. 이 한 번의 Finish가 Pencil을 Claude Code에 자동으로 등록합니다.

1. Claude Code CLI 스위치가 켜져 있는지 본다

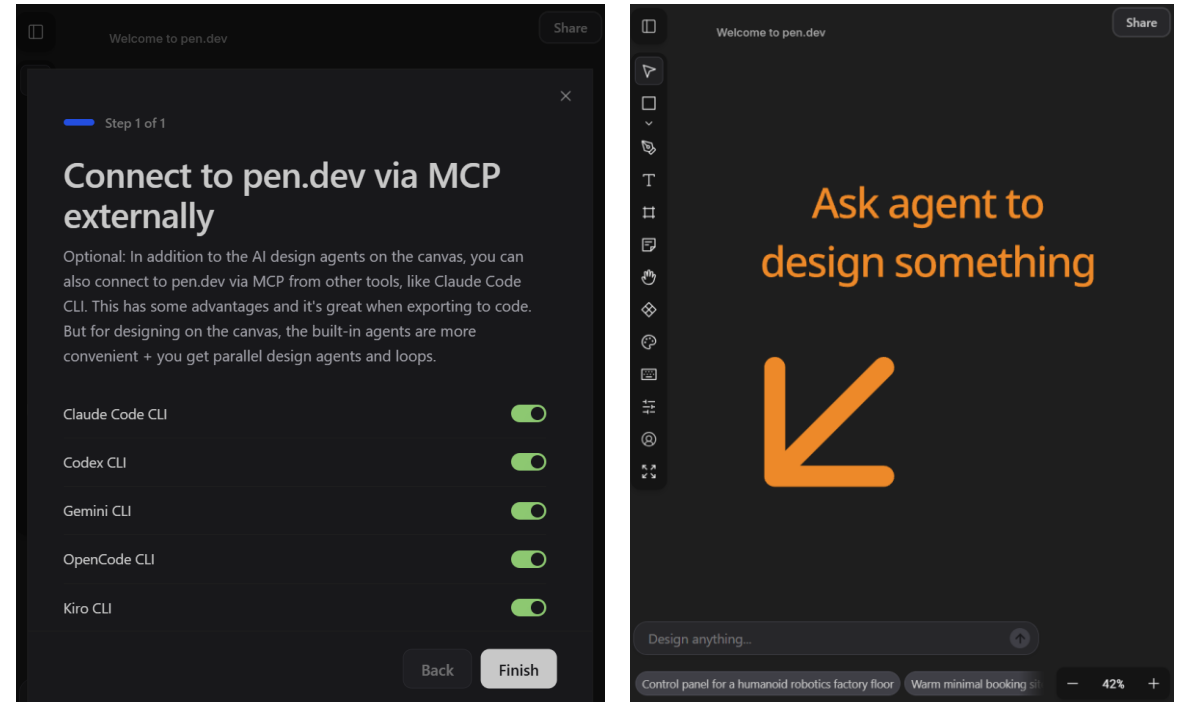
기본값은 켜짐입니다. 나머지(Codex·Gemini 등)는 상관없습니다.

2. Finish

이때 서버의 ~/.claude.json 에 pencil 항목이 자동으로 쓰입니다. 다음 장에서 확인합니다.

3. 캔버스가 뜬다

"Ask agent to design something" 화면. 이 입력창은 쓰지 않습니다. 그리는 것도, 코드에 적용하는 것도 Claude Code가 pencil MCP로 합니다. 캔버스는 결과를 보고 손으로 고치는 자리입니다.



확인 포인트 내화면.pen 캔버스가 열려 있고, 왼쪽에 도형·글자·프레임 도구가 보인다.

참고 이 창을 닫아 버렸으면 왼쪽 Pen.Dev 패널의 Welcome을 열면 다시 나옵니다. ("MCP"는 "Claude Code에 외부 도구를 붙이는 규약"입니다)

서버 터미널에서 세 가지 확인

설치·로그인·허용이 끝났으면 서버 터미널에서 결과를 읽습니다. 셋 다 보이면 연결된 것입니다.

1. 확장이 서버에 있는가

~/.vscode-server/extensions 에 highagency.pencildev-... 폴더.

2. Claude Code 설정에 등록됐는가

~/.claude.json 에 "pencil" 항목. 3/6의 Finish가 쓴 것입니다.

3. Claude Code가 보는가 → claude 를 새로 띄운다

claude mcp list 에 pencil ... ✓ Connected. 설치 전에 띄워 둔 claude 세션은 못 보므로 터미널을 하나 더 열어 claude를 띄웁니다.

확인 포인트 claude mcp list 에 pencil ... ✓ Connected 가 있다.

[서버 터미널] · 설치 → 등록 → 연결 확인

[서버 터미널]

```
$ ls ~/.vscode-server/extensions | grep pencildev  
highagency.pencildev-0.6.71
```

```
$ grep -n pencil ~/.claude.json  
1691:  "pencil": {  
1692:    "command": ".../.pencil/mcp/.../mcp-server-linux-x64"
```

```
$ claude mcp list
```

```
pencil: ... --agent claudeCodeCLI - ✓ Connected  
# claude.ai ... 줄은 내 계정 연동 목록. pencil 줄만 봅니다  
# claude mcp list 는 학기 후반에 배웁니다. 오늘은 결과만 읽습니다
```

안 보이면 ① 창 다시 로드(F1 → Reload Window) ② 내화면.pen 을 다시 연다 ③ claude mcp list 다시. 연결이 안 되면 5/6의 두 프롬프트가 동작하지 않습니다. 그때는 교수 화면(6/6)으로 함께 봅니다.

시안을 코드에 적용해 본다

명세와 시안을 함께 주기만 하면 됩니다. 따로 설정할 것은 없습니다.

1. Claude Code에게 .pen을 그리게 한다

rebuild/요구사항명세서.md에서 첫 화면의 REQ를 고릅니다. 오른쪽 첫 카드대로 claude를 띄우고, 둘째 카드의 첫 프롬프트를 보냅니다. 프레임 안에 있어야 MCP가 읽습니다. 캔버스를 열어 순서와 카드 폭을 손으로 고치면 그것이 기준이 됩니다.

2. 시안을 코드에 적용한다

이어서 둘째 프롬프트를 보냅니다. 시안과 명세가 어긋나면 임의로 정하지 말고 질문하게 했습니다.

3. 결과를 함께 본다

다음 장에서 교수 화면으로 시안과 적용된 화면을 나란히 봅니다. 내 결과가 조금 달라도 됩니다.

확인 포인트 index.html이 시안의 배치대로 바뀌었는지 브라우저에서 본다.

[새 Claude Code 터미널] · 실행

```
$ cd ~/week02/포트폴리오
$ claude
```

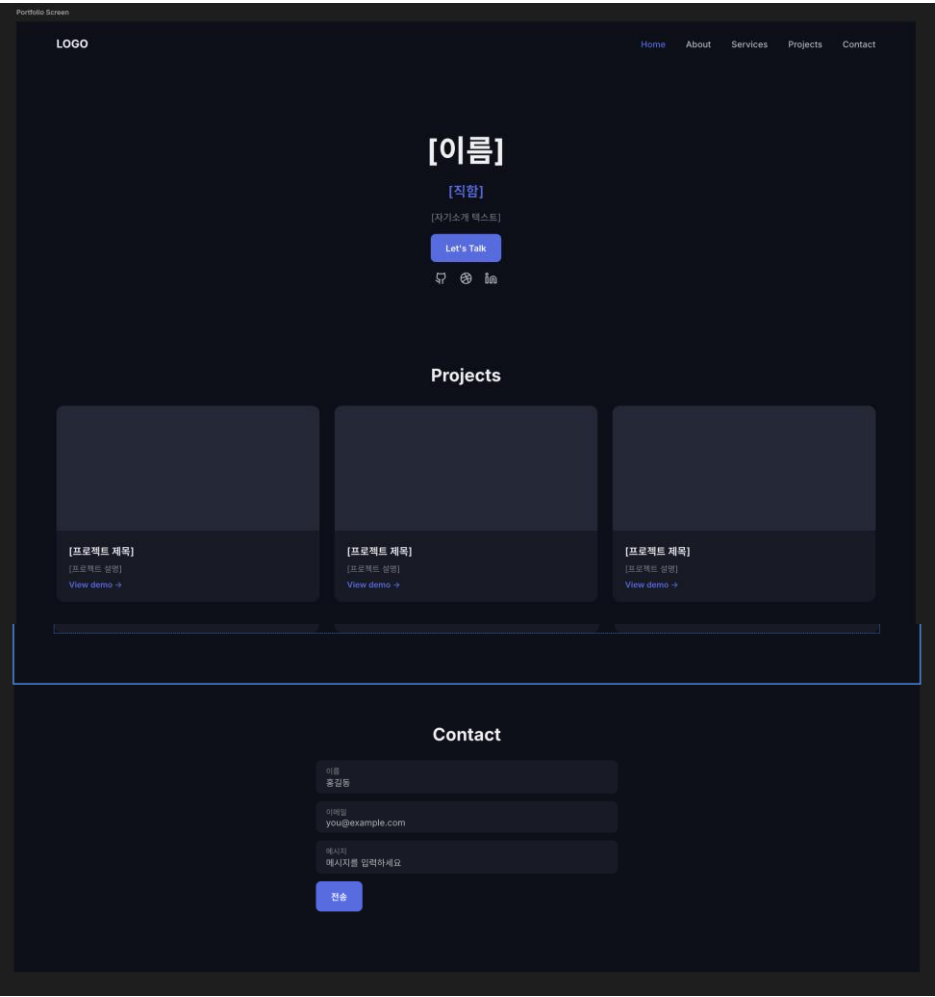
[Claude Code 안] · 두 프롬프트 (그리기 → 적용)

```
> pencil MCP로 rebuild/내화면.pen에 프레임 하나를 만들고,
  헤더 · 소개 · 프로젝트 카드 3열 · 연락처를 위에서 아래로
  배치하라.
> rebuild/내화면.pen의 배치를 rebuild/index.html에 적용
  하라. rebuild/요구사항명세서.md의 REQ와 내 정보는 유지하라.
  시안과 명세가 어긋나면 임의로 정하지 말고 질문하라. 변경한
  파일을 보고하라.
```

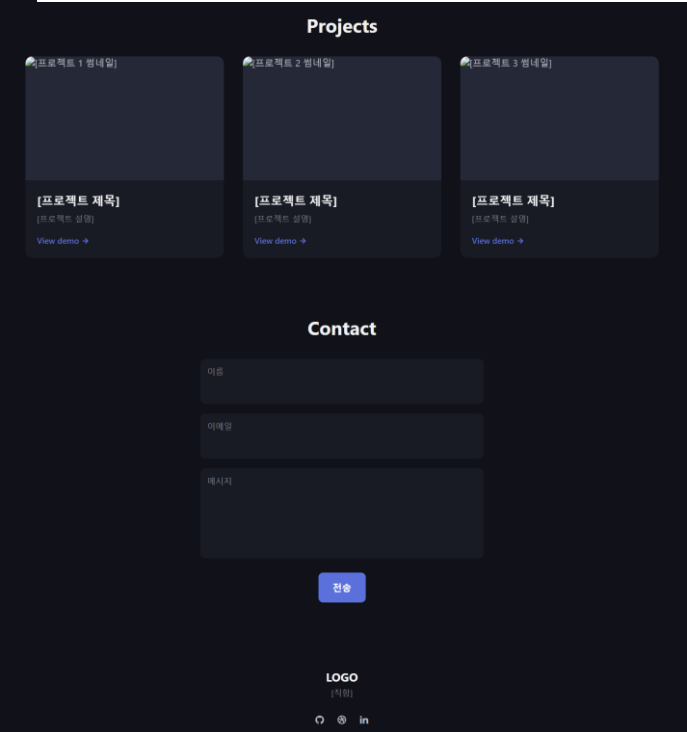
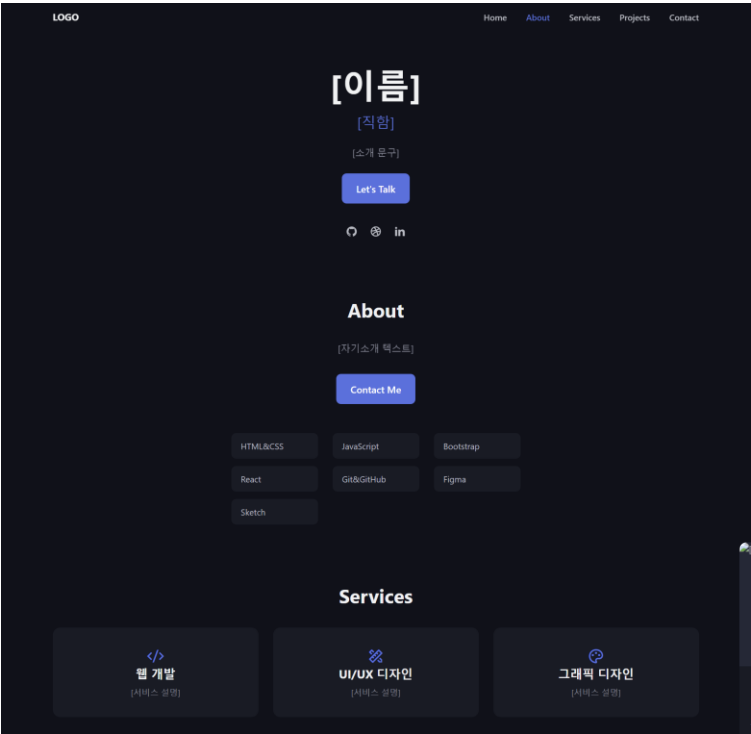
참고 캔버스가 안 열리면 4/6으로 돌아가 확장을 확인합니다. 캔버스의 Design anything 입력창은 서버의 API 키를 보지 못해 아무것도 그려지지 않습니다. 그리는 것도 Claude Code에서 합니다.

화면 명세 · 6/6 · 교수 화면

시안 rebuild/내화면.pen



적용 결과 rebuild/index.html



함께 볼 것

섹션 순서와 카드 배치가 시안대로 왔는가. 기존 기능과 내 정보가 그대로 남았는가.

내 결과가 시안과 조금 달라도 괜찮습니다. 어디가 어떻게 다른지 말할 수 있으면 됩니다.

화면 명세 정리와 실무 활용

오늘 한 것은 한 화면을 그리고, 명세와 함께 주고, 결과를 확인한 것입니다.

1. 왜 그랬나

문장으로 쓰면 해석이 갈리는 배치를 그림으로 정하고, 파일로 남겨 AI가 읽게 하려고.

2. 파일로 남는다는 것

.pen은 캔버스의 프레임·사각형·텍스트를 고유 ID와 타입을 가진 객체 트리
로 적은 JSON 텍스트 파일입니다. 코드와 같은 저장소에 두고 git으로 이력과 변경 병합을 관리합니다. 스크린샷을 주고받는 것과 다른 점이 여기입니다. 디자이너의 레퍼런스도 이미지가 아니라 이 **파일로 받으면**, 필요한 프레임만 골라 쓰고 수정 이력을 diff로 봅니다. **색 · 간격 · 컴포넌트 규칙**이 파일에 적혀 있어, **개발이 규정대로 구현**했는지 대조할 수 있습니다.

3. 도구가 하는 다른 일

Figma에서 복사해 붙일 수 있고, 코드의 컴포넌트를 캔버스로 불러올 수도 있습니다. 색 · 글자 · 여백과 모바일 배치는 오늘 범위 밖입니다.

정리 배치는 그림으로 정하고, 그 결정을 파일로 남겨 사람과 AI가 같은 것을 봅니다. 글 · 사진 · 키는 .pen으로도 정해지지 않아 사람이 넣습니다.

● ● ● 화면 명세 실무 활용

- ① **합의** 말로 정하던 배치를 그 자리에서 그려 파일로
- ② **변경 요청** 「카드 3열로」를 시안 파일로 주면 한 번에
- ③ **인수인계** 명세 (.md)와 시안 (.pen)이 함께 있으면 설명 없이
- ④ **리뷰** 캔버스와 실제 화면을 대조해 통과 · 실패 기록
- ⑤ **레퍼런스** 디자이너 시안을 그림이 아니라 파일로 받아 프레임 단위로 재사용

한계 복잡한 레이아웃은 어긋날 수 있고, .pen은 개발 도구라 비개발자는 열기 어렵습니다.

따로 낼 것은 없고 폴더에 남은 것 그대로

서버 개인 폴더에 오늘 만든 것이 남아 있으면 됩니다.

1. week02/내소재.md

예제 A~D 칸 · 명세 읽기 답 · 생각 거리 · 다음 주에 쓸 예제 하나(바로 다음 정리 활동에서 고릅니다).

2. 포트폴리오/spec/ · rebuild/

예제 D와 화면 명세의 결과, 내화면.pen도 rebuild/ 안에 함께 있습니다.

3. 운동 · 채용의 rebuild/

각각 내 항목이 든 csv와 실행되는 프로그램.

[서버 터미널] · 최종 확인

```
$ cd ~/week02
$ ls 내소재.md 포트폴리오/spec 포트폴리오/rebuild 운동/rebuild 채용/
rebuild
  포트폴리오/spec:      요구사항명세서.md   프롬프트.md
  포트폴리오/rebuild:  index.html   내화면.pen   요구사항명세서.md
  맛집/:              키트 그대로 (교수 데모)
  운동/rebuild · 채용/rebuild:  csv · 프로그램 · 요구사항명세서.md
```

내 프로젝트에 가장 가까운 예제를 하나 골라 둔다

다음 주는 오늘 만든 것 위에서 시작합니다.

1. 하나만 고릅니다

내가 만들고 싶은 것과 구조가 가장 가까운 예제 하나. 오른쪽을 보고 고르면 됩니다.

2. 내소재.md에 한 줄

「가장 가까운 예제: D · 화면이 있고 소개와 목록이 필요해서」 이 정도면 충분합니다.

3. 다음 주에 하는 일

고른 예제의 명세서를 그대로 두고, 프롬프트로 내 프로젝트에 맞게 바꿔 시작합니다. 오늘 만든 파일이 그때 재료가 됩니다.

확인 포인트 내소재.md에 고른 예제와 이유 한 줄이 있다.

● ● ● 고를 때 참고

- | | |
|---------|-------------------------|
| A 수영 강좌 | URL 하나 → 목록과 필터가 있는 CLI |
| B 채용 공고 | 같은 구조, 재료만 다름 |
| C 지도 관찰 | 눌러야 근거가 나오는 서비스 (데모) |
| D 포트폴리오 | 화면이 있는 정적 웹페이지 |

참고 고르기 어려우면 D(내 포트폴리오)로 두면 됩니다.

명세가 있으면 무엇이 달라지는가

오늘은 프로그램이 작아 명세가 없어도 돌아갔습니다. 실무에서 갈리는 지점은 네 곳입니다.

1. 인수인계

담당자가 바뀌면 코드를 읽어 의도를 역산해야 합니다. **명세가 있으면 무엇을 만들기로 했는지가 문서로 남습니다.**

2. 변경과 유지보수

「왜 이렇게 동작하나」의 근거가 요구사항입니다. 근거가 없으면 고칠 때마다 추측이 쌓입니다.

3. 검수와 계약

외주 · 발주에서는 **요구사항 명세가 검수 기준**입니다. 문서가 없으면 무엇을 완료로 볼지 합의할 수 없습니다.

4. 테스트

테스트는 **요구사항에서** 나옵니다. 확인 방법이 없는 요구는 다음 주에 테스트가 되지 않습니다.

● ● ● 규제 산업에서는 문서가 규격

의료기기 IEC 62304 · 자동차 ISO 26262 · 항공 DO-178C
요구사항과 코드 · 테스트 사이의 추적성을 문서로 요구합니다.

● ● ● AI로 만들 때

명세가 곧 입력입니다. 같은 폴더에서 같은 프롬프트를 써도
명세가 얇으면 AI가 빈칸을 스스로 채웁니다.
오늘 A~D에서 본 「추측」 표시가 그 자리였습니다.

● ● ● 화면 명세 도구는 여러 가지

수업에서는 Pencil(.pen)을 썼습니다. Figma · Penpot · Excalidraw로 그린 시안도
같은 역할을 합니다. 도구보다 배치 결정을 파일로 남기는 것이 핵심입니다.

기억할 것 명세의 품질이 제품 품질의 상한입니다. 문장이 기능을 정하고, 그림이 배치를 정합니다. 명세 = 문장 + 화면.

NEXT WEEK

다음 주 맛보기: TDD와 첫 배포

Red

실패하는 테스트를 먼저 씁니다. 요구사항이 테스트가 됩니다.

Green

테스트를 통과하는 최소한의 코드를 만듭니다.

Refactor

통과를 유지한 채 코드를 다듬습니다.

로컬 vs 배포

내 컴퓨터에서 되던 것이 배포에서 왜 문제를 일으키는지 실제 사례로 확인합니다.

첫 배포

만든 프로그램을 실제 인터넷 주소로 배포해 봅니다.

TDD(Test-Driven Development)는 테스트가 개발을 이끄는 방식입니다. 오늘 만든 rebuild/ 중 하나의 최신 명세서와 발전 버전이 다음 주의 입력물입니다.

오늘 정리 · 다음 주

오늘 배운 것

새 기능은 로드맵 위에 놓는다

프롬프트 → 컨텍스트 → 하네스 → 루프.

단계는 압축될 뿐 사라지지 않는다

건너뛴 단계는 반드시 문제로 돌아온다.

코드가 있든 화면뿐이든 절차는 같다

예제 넷: 준비와 첫 프롬프트만 다르고 나머지는 같은 문장.

명세를 뽑고, 읽고, 명세만으로 다시 만든다

달라진 부분이 명세의 빈틈. 읽고 고쳐야 내 명세.

제출물

week02의 내소재.md · 포트폴리오/spec · 세 예제(포트폴리오 · 운동 · 채용)의 rebuild
/. 별도 업로드 없음.

다음 주

TDD와 첫 배포

로컬과 배포의 차이에서 시작해, 배포 전에 품질을 보장하는 TDD를 실습합니다. 오늘 만든 rebuild/ 중 하나가 입력물입니다.